



Introduction to Deep Learning with Keras and TensorFlow



TensorFlow

TensorFlow™ is an open-source software library for numerical computation using data flow graphs (originally developed by the Google Brain Team). <https://www.tensorflow.org/>

- Python API
- flexible architecture allows deploying computation to one or more CPUs or GPUs in a desktop, server, or mobile device with a single API
- could be used on various platforms from Raspberry Pi, Android, Windows, iOS, Linux to server farms
- Many big companies like DeepMind, Uber, AirBnB, or Dropbox have all decided to leverage this framework.
- is supported by large community, therefore, a lot of documentation, guidelines and tutorials are freely available in the web.

TensorBoard is a suite of web applications for inspecting and understanding your TensorFlow runs and graphs. Use TensorBoard to:

- visualize TensorFlow graph
- plot quantitative metrics about the execution of your graph
- show additional data like images that pass through it

https://www.tensorflow.org/tensorboard/get_started

<https://github.com/tensorflow/tensorboard>

Some basics...

CS230 tutorial: TensorFlow for Deep Learning Research

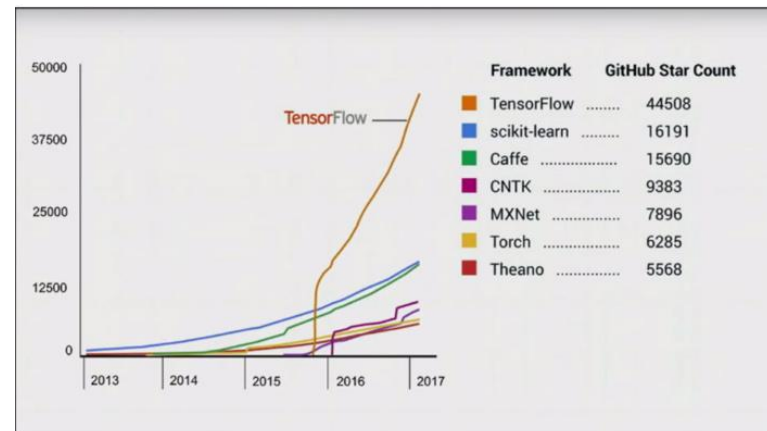
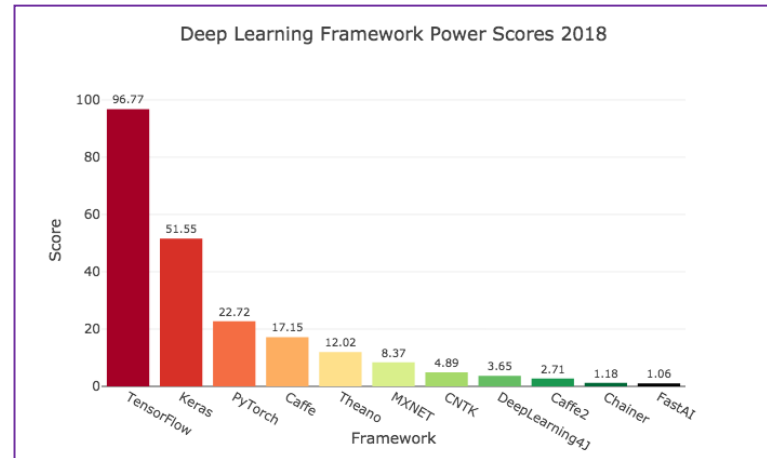
<https://cs230.stanford.edu/blog/tensorflow/>

Video tutorial:

- https://www.youtube.com/watch?v=QPDsEtUK_D4

To become familiar with TensorFlow, check the official low-level tutorial:

<https://www.tensorflow.org/guide/basics>





TensorFlow

- ❑ *Install TensorFlow following the instructions* (<https://www.tensorflow.org/install/>)

You may install with native pip, or via Anaconda environment.

Check installation running the code:

```
python
>>> import tensorflow as tf
>>> print("TensorFlow version:", tf.__version__)
>>> hello = tf.constant('Hello, TensorFlow!')
>>> print(hello.numpy())
```

```
TensorFlow version: x.x.x
'Hello, TensorFlow!'
```

- ❑ *Docker image of Tensorflow*

- *Install **Docker** on your platform of choice* (<https://www.docker.com/pricing/#/download>, <https://docs.docker.com/get-started/get-docker/>).
- *Install **Docker image for Tensorflow*** (<https://hub.docker.com/r/tensorflow/tensorflow/>)

- ❑ *Use **Jupyter Notebook** - an open-source web application that allows you to create and share documents that contain live code, equations, visualizations and narrative text.* (<https://jupyter.org/>)

- ❑ *Use **Google Colaboratory**: Google's free cloud service for AI developers using Keras, Tensorflow and PyTorch.* (<https://colab.research.google.com/>).

Relevant tutorial: <https://colab.research.google.com/notebooks/welcome.ipynb>
<https://www.tensorflow.org/tutorials>

- ❑ *Take a look at **Interactive Python tutorials**:* <https://www.learnpython.org/>



TensorFlow

Tensor in a deep learning framework is the data structure used by machine learning systems. It is a container for numerical data that could be present by three main attributes:

- **rank** refers to the tensor's number of axes.
- **shape** refers to the number of dimensions along each axis. E.g. (2,2) or (2,5,10), etc.
- **data type** refers to the type of data contained in tensor (e.g. float32, float64, uint8, int32, int64, etc.).

Some common tensor representations:

- **Vectors:** 1D — (features)
- **Sequences:** 2D — (timesteps, features)
- **Images:** 3D — (height, width, channels)
- **Videos:** 4D — (frames, height, width, channels)

Corresponding batches of e.g. images or videos will be present by 1 more dimensional tensors:

- **4D** for images (samples, height, width, channels)
- **5D** for videos (samples, frames, height, width, channels)

Tensor could be considered as n-dimensional matrix:

0-d tensor: scalar (number)

1-d tensor: vector

2-d tensor: matrix

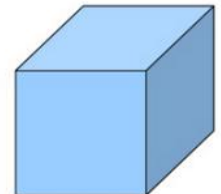
Etc.



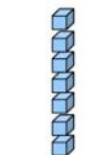
1d-tensor



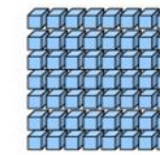
2d-tensor



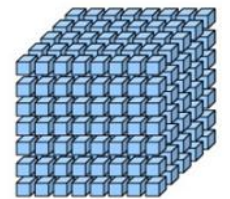
3d-tensor



4d-tensor



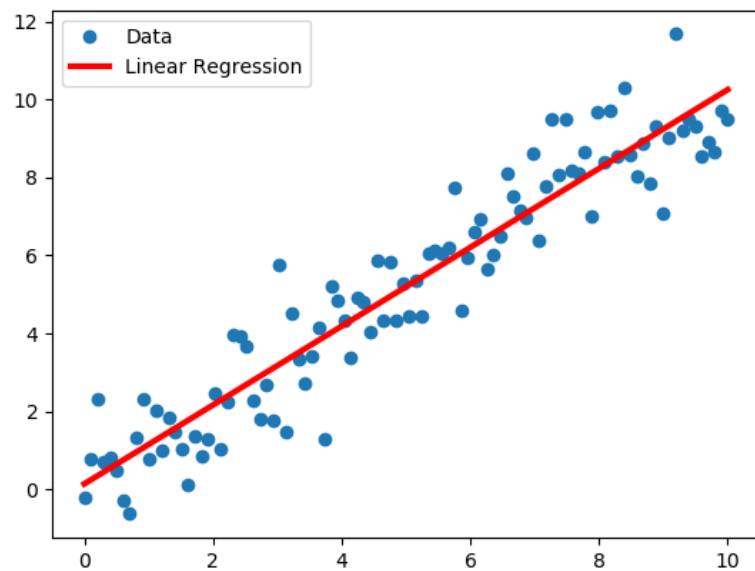
5d-tensor



6d-tensor

Relevant links:

<https://medium.com/datadriveninvestor/what-is-the-tensor-in-deep-learning-77c2af7224a1>



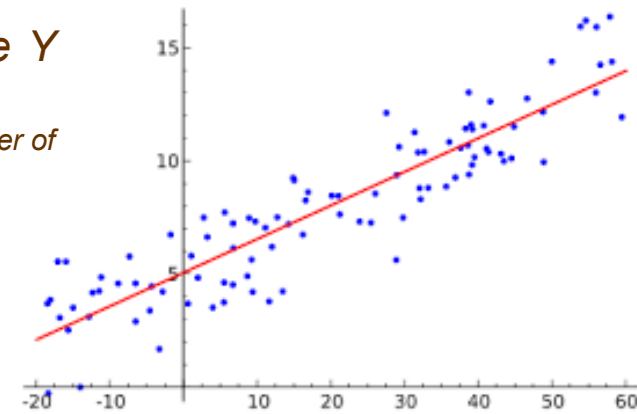
Linear Regression



Linear Regression

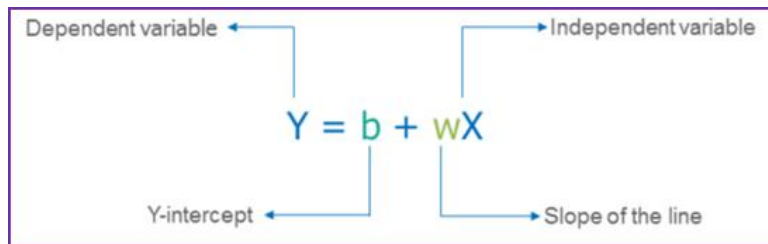
Model relationship between a scalar dependent variable Y and independent variables X .

(e.g. dependence of salary level from age, or amount of theft incidents from number of fire cases, or house price from distance to city center, etc.)

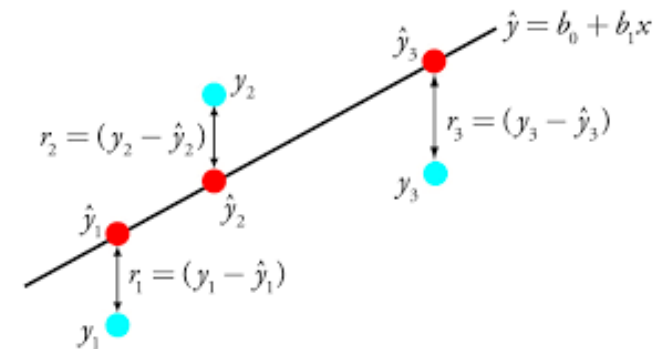


Model: Multiply each **feature** by a **weight** and add them up. Add an **intercept** to get the final estimate.

$$y = w_0x_0 + w_1x_1 + \dots + w_nx_n + b$$



Cost function: $(Y - Y_{\text{predicted}})^2$



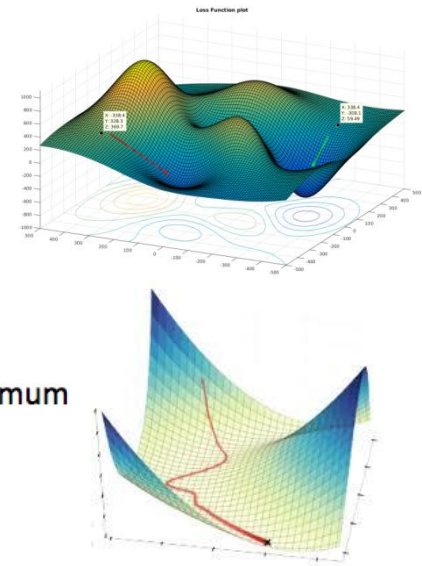
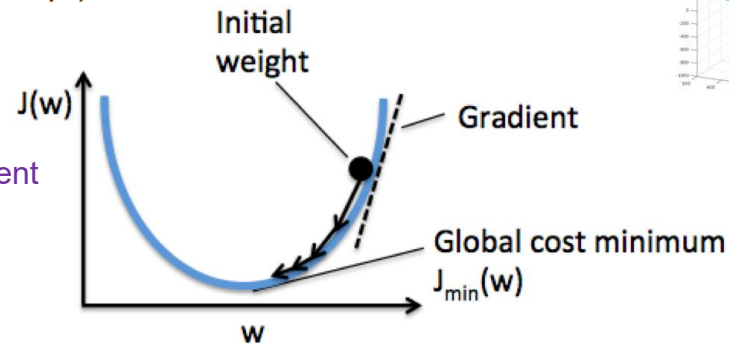


Linear Regression

Optimization: Holding X and Y constant, adjust **parameters** (weights and intercept) to minimize **cost**.

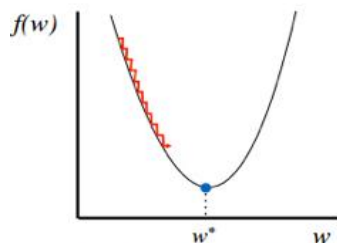
Gradient descent:

https://en.wikipedia.org/wiki/Gradient_descent



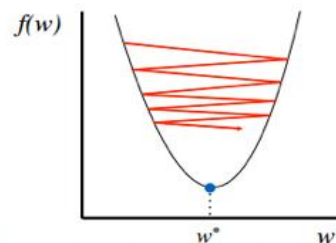
Hyper-parameter **Learning rate** allows us to manipulate with our learning steps...

Undershoot



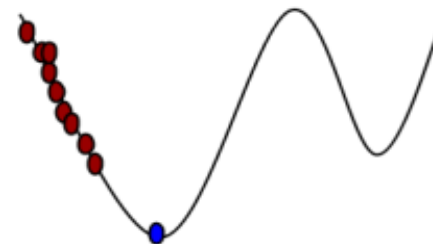
Too small: converge very slowly

Overshoot



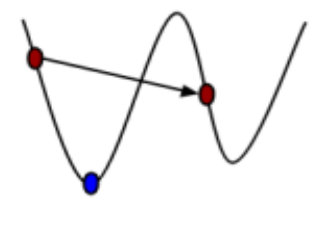
Too big: overshoot and even diverge

Undershoot



very small learning rate needs lots of steps

Overshoot



too big learning rate: missed the minimum

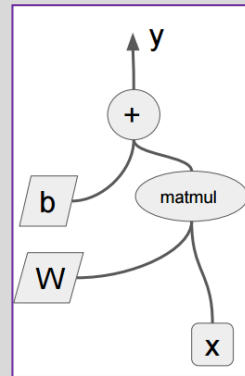
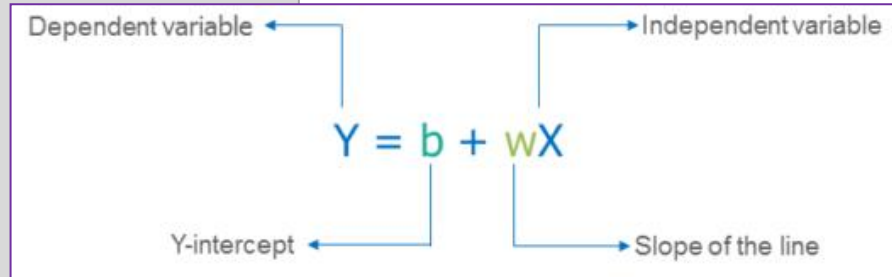


Linear Regression

Simple Linear Regressor

```
import tensorflow as tf

# Model parameters
W = tf.Variable([.3], dtype=tf.float32)
b = tf.Variable[-.3], dtype=tf.float32)
# Model input and output
x = tf.placeholder(tf.float32)
linear_model = W*x + b
y = tf.placeholder(tf.float32)
# loss
loss = tf.reduce_sum(tf.square(linear_model - y)) # sum of the squares
# optimizer
optimizer = tf.train.GradientDescentOptimizer(0.01)
train = optimizer.minimize(loss)
# training data
x_train = [1, 2, 3, 4]
y_train = [0, -1, -2, -3]
# training loop
init = tf.global_variables_initializer()
sess = tf.Session()
sess.run(init) # reset values to wrong
for i in range(1000):
    sess.run(train, {x: x_train, y: y_train})
# evaluate training accuracy
curr_W, curr_b, curr_loss = sess.run([W, b, loss], {x: x_train, y: y_train})
print("W: %s b: %s loss: %s"%(curr_W, curr_b, curr_loss))
# predict new values
x_predict = [-1, 0, 1, 2]
predicted_values = [(W*x + b).eval(session=sess) for x in x_predict]
print(predicted_values)
```



List of optimizers in TF:

- `tf.train.GradientDescentOptimizer`
- `tf.train.AdagradOptimizer`
- `tf.train.MomentumOptimizer`
- `tf.train.AdamOptimizer`
- `tf.train.ProximalGradientDescentOptimizer`
- `tf.train.ProximalAdagradOptimizer`
- `tf.train.RMSPropOptimizer`
- ...



Linear Regression generalized

```
...
N = 3      # is number of variables
# Placeholders
X = tf.placeholder(tf.float32, [None, N])
Y = tf.placeholder(tf.float32, [None, 1])
# Parameters/Variables
W = tf.get_variable("weights", [N, 1],
                    initializer=tf.random_normal_initializer())
b = tf.get_variable("intercept", [1],
                    initializer=tf.constant_initializer(0))
learning_rate = 0.01
NUM_EPOCHS = 10
BATCH_SIZE = 50
X_train, Y_train, X_test, Y_test = getData() # function should be implemented
# Model
Y_model = tf.matmul(X, W) + b
# Cost function (loss)
loss = tf.reduce_mean(tf.square(Y_model - Y)) # mean(average) of the squares
# Optimization
optimizer = tf.train.GradientDescentOptimizer(learning_rate).minimize(loss)
# -----
with tf.Session() as sess:
    # initialize variables
    sess.run(tf.global_variables_initializer())
    # Train
    for _ in range(NUM_EPOCHS):
        for X_batch, Y_batch in get_minibatches(X_train, Y_train, BATCH_SIZE): # function should be implemented
            sess.run(optimizer, feed_dict={X: X_batch, Y: Y_batch})
    # Test
    Y_predicted = sess.run(Y_model, feed_dict = {X: X_test})
    squared_err = tf.reduce_mean(tf.square(Y_test - Y_predicted))
```

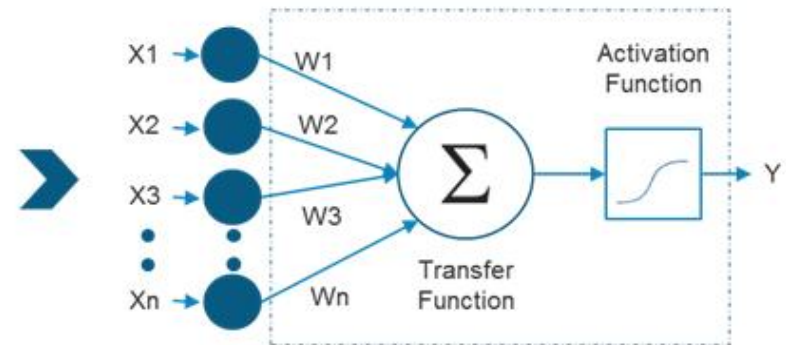
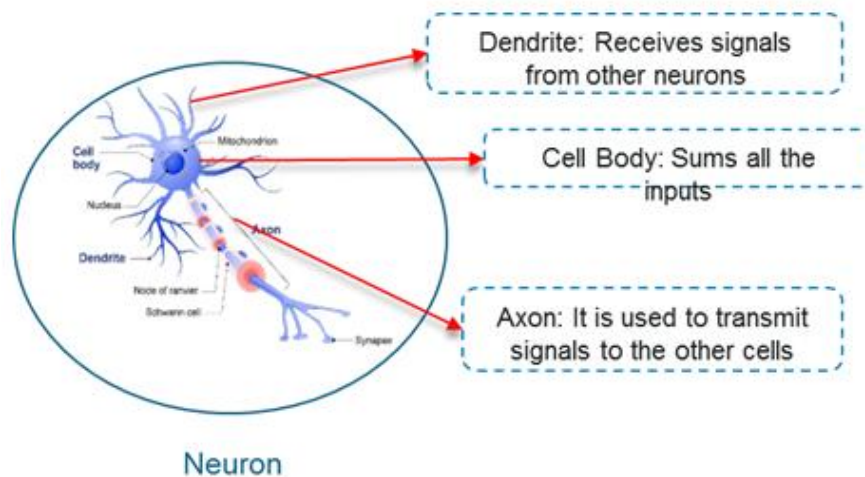
$$y = w_0x_0 + w_1x_1 + \dots + w_nx_n + b$$

Linear Regression ? Neural Network

Linear Regression is a simplest Neural Network...

Aiming at re-engineering a human brain, **Deep Learning** studies the basic unit of a brain - a brain cell or a **neuron**...

Perceptron – is an artificial neuron developed in analogy to a biological neuron.



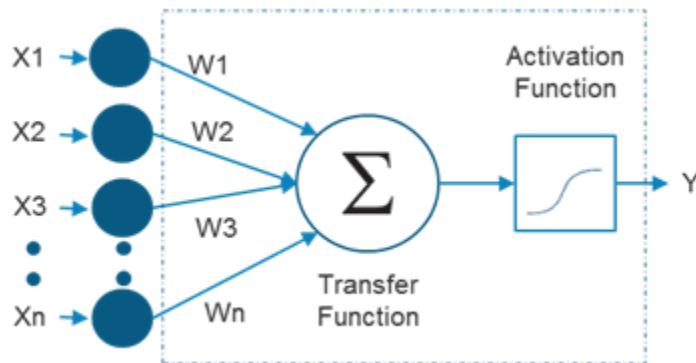
Schematic for a neuron in a neural net

Relevant links:

Activation function

Been fed with a lot of information simultaneously our brain tries to distinguish and classify the information between useful and not-so-useful...

Activation functions help the network do the segregation. They basically decide whether a neuron should be activated or not. They help the network use the useful information and suppress the irrelevant data points.



- The neurons do a linear transformation on the input by the weights and biases.
- The non-linear transformation is done by the activation function.

$$Y = \text{Activation}(\Sigma(\text{weight} * \text{input}) + \text{bias})$$

- Activation functions make the back-propagation possible since the gradients are supplied along with the error to update the weights and biases. Without the differentiable non-linear function, this would not be possible.

Activation function

○ Binary Step and Linear Functions

- the derivatives are "0" and "constant", meaning that every time we do a back propagation, the gradient would be the same.

○ Sigmoid or Logistic

- isn't zero centered that makes optimization harder
- vanishing gradient problem

○ Tanh – Hyperbolic tangent

- vanishing gradient problem

○ ReLU – Rectified linear units

- it should only be used within Hidden layers (output layer should use **Softmax** function for a Classification and simply use a **linear** function for a regression problem).
- some gradients can be fragile during training and can die resulting in Dead Neurons

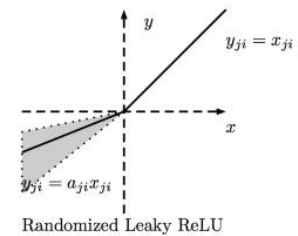
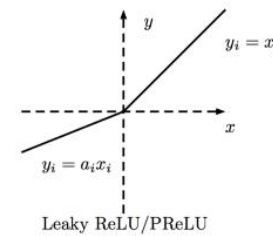
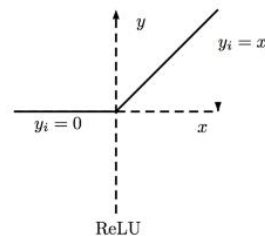
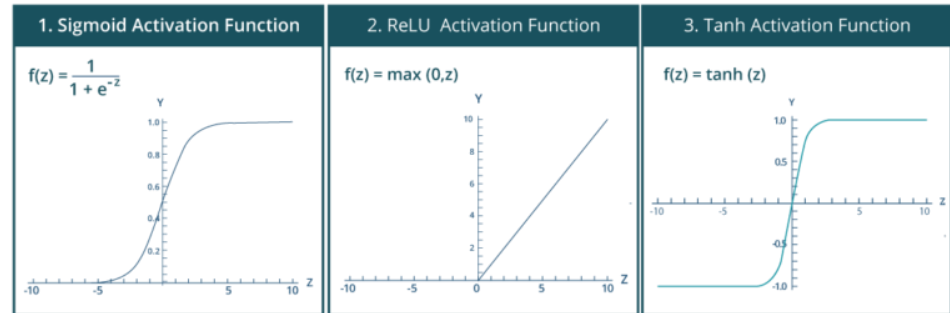
○ Leaky ReLU (Parameteric PReLU, Randomized RReLU)

- it should only be used within Hidden layers

○ Maxout (forms both ReLu and Leaky ReLu)

Relevant links:

https://en.wikipedia.org/wiki/Activation_function



The **Softmax** function is a sigmoid type of function but is handy to handle multiple classes classification problems. Sigmoid helps to classify between two classes, Softmax helps to attain the probabilities to define the class of each input.

Use ReLu (applied to the hidden layers only) and, if the Model suffers form dead neurons during training, use Leaky ReLu or Maxout function.



Keras is a high-level neural networks API, written in Python and capable of running on top of **TensorFlow**, **CNTK**, or **Theano**. (<https://keras.io/>)

(https://keras.io/getting_started/intro_to_keras_for_engineers/) (https://www.tensorflow.org/guide/keras/sequential_model)

It was developed with a focus on enabling fast experimentation. Being able to go from idea to result with the least possible delay is key to doing good research.

- Allows for easy and fast prototyping (through user friendliness, modularity, and extensibility).
- Supports both convolutional networks and recurrent networks, as well as combinations of the two.
- Runs seamlessly on CPU and GPU.

Keras is compatible with: Python 2.x-3.x.

The core data structure of Keras is a **model**, a way to organize layers. The simplest type of model is the **Sequential model**, a linear stack of layers. For more complex architectures, you should use the Keras **Functional API**, which allows to build arbitrary graphs of layers. Finally, you may use fully-customizable **model subclassing** to implement own custom forward-pass of the model...

Example:

```
# example of a model definition with the sequential api
from tensorflow import keras
from keras.models import Sequential
from keras.layers import Dense
# define the model
model = Sequential()
model.add(Dense(units=64, activation='relu', input_dim=100))
model.add(Dense(units=1))
...
```

```
# example of a model definition with the functional api
from tensorflow.keras import Model
from tensorflow.keras import Input
from tensorflow.keras.layers import Dense
# define the layers
x_in = Input(shape=(100,))
x = Dense(64)(x_in)
x_out = Dense(1)(x)
# define the model
model = Model(inputs=x_in, outputs=x_out)
...
```

Relevant links:

https://www.tensorflow.org/guide/keras/sequential_model

<https://github.com/keras-team/keras>

<https://www.pyimagesearch.com/2019/10/28/3-ways-to-create-a-keras-model-with-tensorflow-2-0-sequential-functional-and-model-subclassing/>

Layer

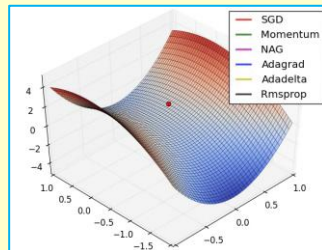
- Dense
- Convolutional: Conv1D, Conv2D, Conv3D
- Pooling: ...
- Recurrent: RNN, GRU, LSTM
- Flatten
- Activation
- Dropout

Losses

- `mean_squared_error`
- `mean_squared_error(y_true, y_pred)`
- `mean_absolute_error(y_true, y_pred)`
- `mean_absolute_percentage_error(y_true, y_pred)`
- `mean_squared_logarithmic_error(y_true, y_pred)`
- `categorical_crossentropy(y_true, y_pred)`

Optimizers

- `SGD()`
- `RMSprop()`
- `Adagrad()`
- `Adadelta()`
- `Adam()`



Metrics

- `accuracy`
- `binary_accuracy`
- `binary_accuracy(y_true, y_pred)`
- `categorical_accuracy(y_true, y_pred)`
- `top_k_categorical_accuracy(y_true, y_pred, k=5)`

Activations

- `softmax`
- `relu`
- `tanh`
- `sigmoid`
- `hard_sigmoid`
- `Linear`

Model compilation

- **Loss function.** This is the objective that the model will try to minimize. It can be the string identifier of an existing loss function (such as `categorical_crossentropy` or `mse`), or it can be an objective function.
- **Optimizer.** This could be the string identifier of an existing optimizer (such as `rmsprop` or `adagrad`), or an instance of the `Optimizer` class.
- **List of metrics**

```
model.compile(loss='mean_squared_error', optimizer='sgd',
              metrics=[metrics.mae, metrics.categorical_accuracy])
```

```
model.compile(loss='categorical_crossentropy', optimizer='sgd',
              metrics=['accuracy'])
```

```
model.compile(loss=keras.losses.categorical_crossentropy,
              optimizer=keras.optimizers.SGD(lr=0.01, momentum=0.9, nesterov=True),
              metrics=[keras.metrics.SparseCategoricalAccuracy(name="acc")])
```

Model training Keras models are trained on Numpy arrays of input data and labels, typically using `fit` function.

```
history = model.fit(X_train, Y_train, batch_size=128, epochs=8)
```

Model evaluation

```
loss_and_metrics = model.evaluate(x_test, y_test, batch_size=128)
```

Model prediction

```
prediction = model.predict(x_test, batch_size=128)
```

K Regression problem

Boston house price - The dataset describes 13 numerical properties of houses in Boston suburbs and is concerned with modeling the price of houses in those suburbs in thousands of dollars.

Dataset: <https://raw.githubusercontent.com/jbrownlee/Datasets/master/housing.data> (save the content as housing.csv file)

```
import pandas as pd
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.model_selection import train_test_split

# data preprocessing
dataframe = pd.read_csv("housing.csv", sep='\s+', header=None)
dataset = dataframe.values
X = dataset[:,0:13]
Y = dataset[:,13]

# create model
model = Sequential()
model.add(Dense(1, input_dim=13, activation='relu'))
# you may play with different network architectures (adding new layers, and changing their amount of nodes)

# compile model
model.compile(loss='mean_squared_error', optimizer='adam', metrics = ['accuracy'])
# split the data
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = 0.2, random_state = 0)
# fit the model
model.fit(X_train, Y_train, epochs=100, verbose=0)
# evaluate the model
mse_value, acc = model.evaluate(X_test, Y_test, verbose=0)
print('Evaluation MSE: '+str(mse_value))
# predict the value
Y_pred = model.predict(X_test[0].reshape(1, -1))
print(Y_pred[0][0])
```

Source: <https://machinelearningmastery.com/regression-tutorial-keras-deep-learning-library-python/>
<https://www.tensorflow.org/tutorials/keras/regression>

Public datasets...

- ❑ **Kaggle Datasets** (<https://www.kaggle.com/datasets>)
- ❑ **Hugging Face Datasets** (<https://huggingface.co/datasets>)
- ❑ **UCI Machine Learning Repository Datasets** (<https://archive.ics.uci.edu/>)
- ❑ **Google Dataset Search** (<https://datasetsearch.research.google.com/>)
- ❑ **Awesome Public Datasets** (GitHub) (<https://github.com/awesomedata/awesome-public-datasets>)
- ❑ **Harvard Dataverse** (<https://dataverse.harvard.edu/>)
- ❑ **Papers with code datasets** (<https://github.com/paperswithcode/paperswithcode-data/>)
- ❑ **Amazon Open Data Registry** (<https://registry.opendata.aws/>)
- ❑ **Azure Open Datasets** (<https://learn.microsoft.com/en-us/azure/open-datasets/dataset-catalog/>)
- ❑ **Data.gov - Government's Open Data** (<https://data.gov/>)
- ❑ **Google Open Images / Roboflow** (<https://storage.googleapis.com/openimages/web/index.html/>)
- ❑ **Nasdaq Data Link** (<https://www.quantconnect.com/docs/v2/writing-algorithms/datasets/nasdaq/data-link/>)

Datasets withing Python Libraries:

- `sklearn.datasets`
- `tensorflow_datasets`
- `pytorch_datasets`

Relevant links:

- <https://medium.com/datadriveninvestor/the-50-best-public-datasets-for-machine-learning-d80e9f030279>
- https://en.wikipedia.org/wiki/List_of_datasets_for_machine_learning_research
- <https://365datascience.com/trending/public-datasets-machine-learning/>



Lecture 2: Introduction to Neural Networks with TensorFlow (*Classification*)

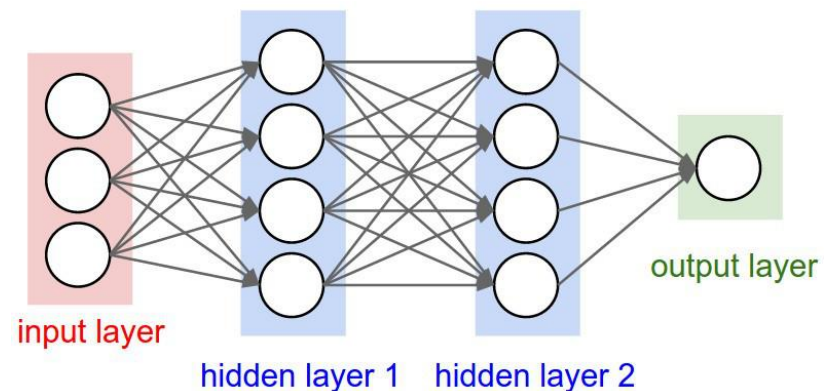
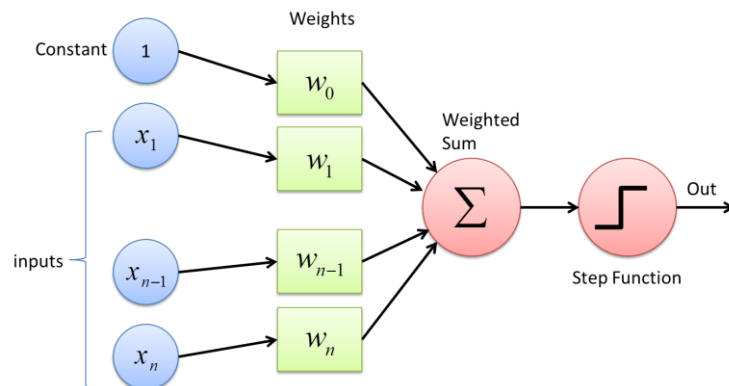
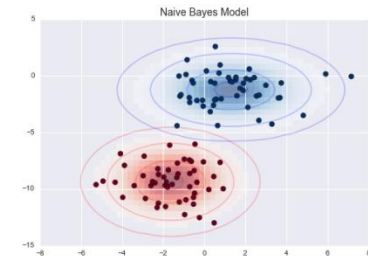
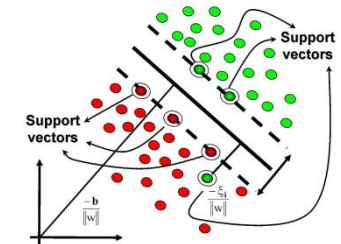
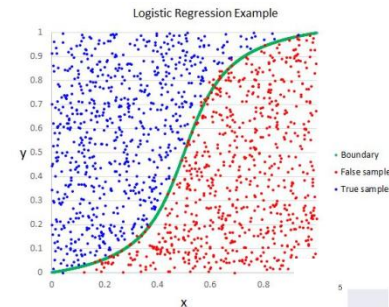
Classification

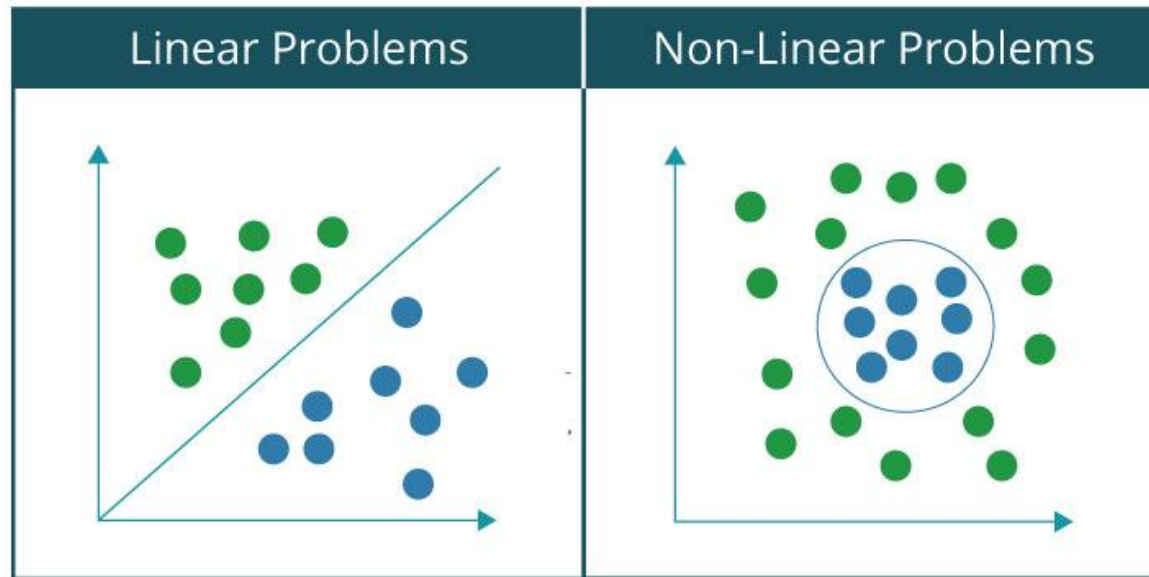
Classification – is the process of categorizing a group of objects...

Existing classifiers are:

- Logistic Regression,
- Support Vector Machines (SVM),
- Decision Tree,
- Naive Bayes,
- Neural Networks,
- ...

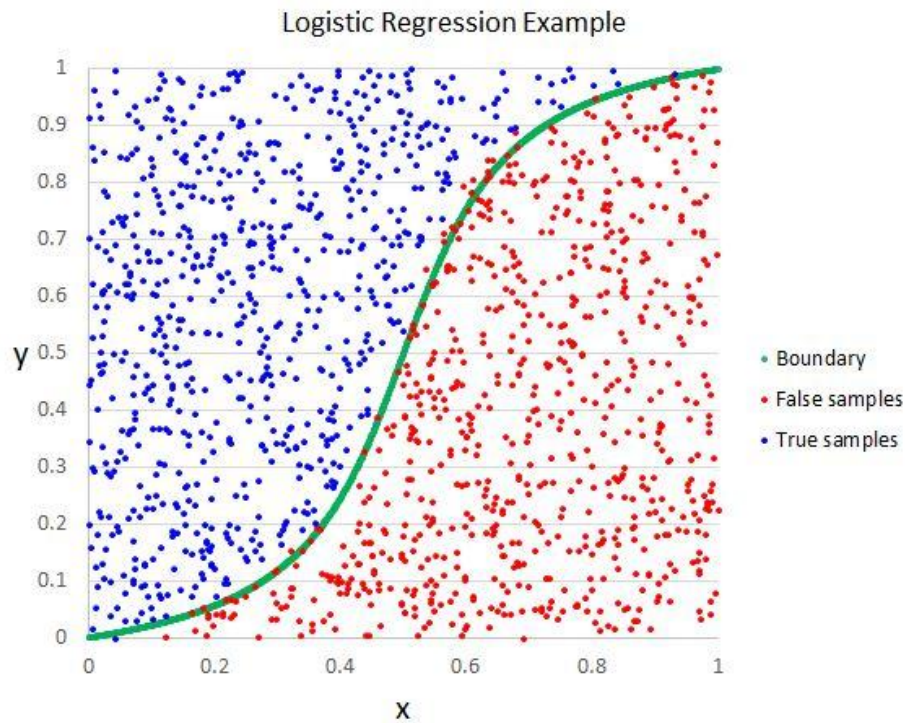
In comparison to other classifiers, when the patterns get complex, neural nets start to outperform all of their competition. The main **advantage of DNN is automated feature extraction** (those that are not specified/set explicitly).





Relevant links:

<https://www.edureka.co/blog/perceptron-learning-algorithm/>



Logistic Regression

Perceptron as a Linear Classifier

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
from datetime import datetime
import numpy as np
%load_ext tensorboard

#input1, input2 and bias
X_train=np.array([[1., 1.,1], [1., 0,1], [0, 1.,1], [0, 0,1]])
#output
y_train = np.array( [[1.], [0], [0], [0]])

logdir = "logs/scalars/" + datetime.now().strftime("%Y%m%d-%H%M%S")
tensorboard_callback = tf.keras.callbacks.TensorBoard(log_dir=logdir)
# define model
model = Sequential()
model.add(Dense(1, activation='sigmoid'))
# compile the model
model.compile(optimizer='adam', loss='binary_crossentropy',
              metrics=['accuracy'])

# fit the model
model.fit(X_train, y_train, epochs=300, batch_size=4, verbose=1,
          callbacks=[tensorboard_callback])

# evaluate the model
loss, acc = model.evaluate(X_train, y_train, verbose=0)
print('Test Accuracy: %.3f % acc)

# make a prediction
row = np.array( [0,1,1])
yhat = model.predict(row.reshape(1,3)) [0][0]
print(f"Predicted: {yhat:.3f}")
print("Class: 0") if yhat < 0.5 else print("Class: 1")
%tensorboard --logdir logs/scalars
```



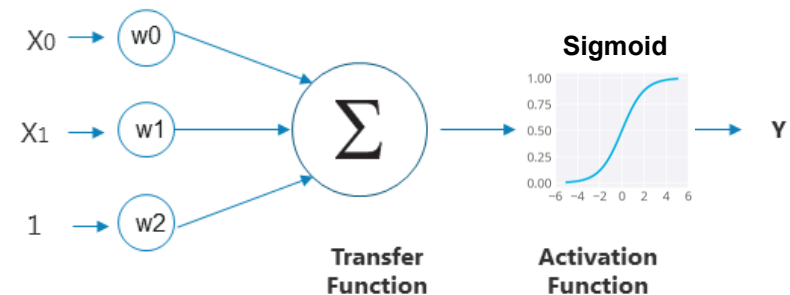
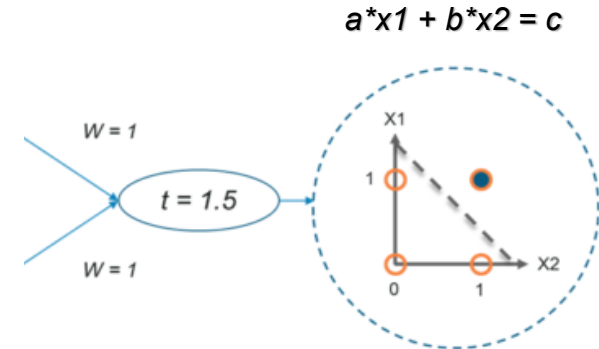
Linear Classifier

binary logistic regression

AND Gate



x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1



Mathematically, Binary Cross-Entropy (BCE) is defined as:

$$BCE = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

where:

- N is the number of observations
- y_i is the actual binary label (0 or 1) of the i^{th} observation.
- p_i is the predicted probability of the i^{th} observation being in class 1.

Classification on MNIST dataset

Each image is a 28x28 array, flattened out to be a 1-d tensor of size 784

MNIST.train: 55,000 examples

MNIST.validation: 5,000 examples

MNIST.test: 10,000 examples

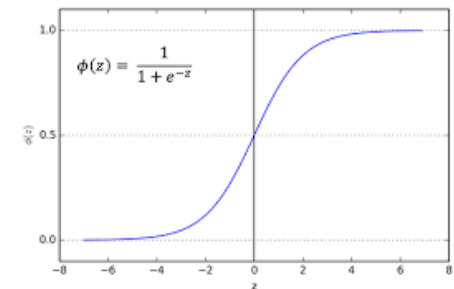


Model: take a **weighted sum** of the features and add a **bias** to get the **logit**. Convert the logit to a **probability** via the **logistic-sigmoid function**.

$$\text{logits} = X * W + b$$

$$Y_{\text{predicted}} = \text{softmax}(\text{logits})$$

$$\text{loss} = \text{cross_entropy}(Y, Y_{\text{predicted}}) \quad , \text{ where } Y \text{ is a one-hot vector}$$



MNIST Dataset: https://en.wikipedia.org/wiki/MNIST_database; <http://yann.lecun.com/exdb/mnist/>

MNIST Dataset in csv format:

One-Hot representation

One-Hot is a way to represent categorical data. It is a group of bits among which the legal combinations of values are only those with a single high (1) bit and all the others low (0). A similar implementation in which all bits are '1' except one '0' is sometimes called **one-cold**.

Label Encoding

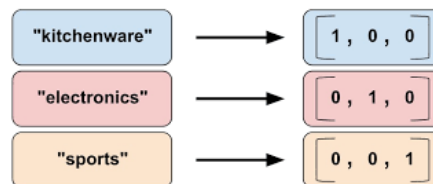
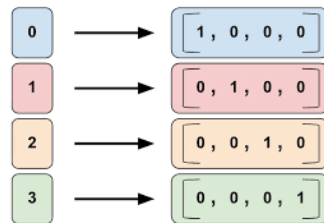
Food Name	Categorical #	Calories
Apple	1	95
Chicken	2	231
Broccoli	3	50



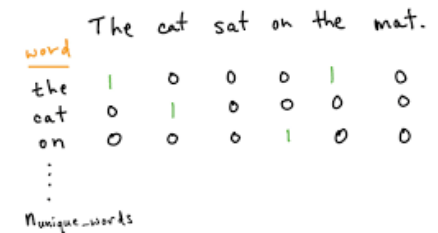
One Hot Encoding

Apple	Chicken	Broccoli	Calories
1	0	0	95
0	1	0	231
0	0	1	50

Examples

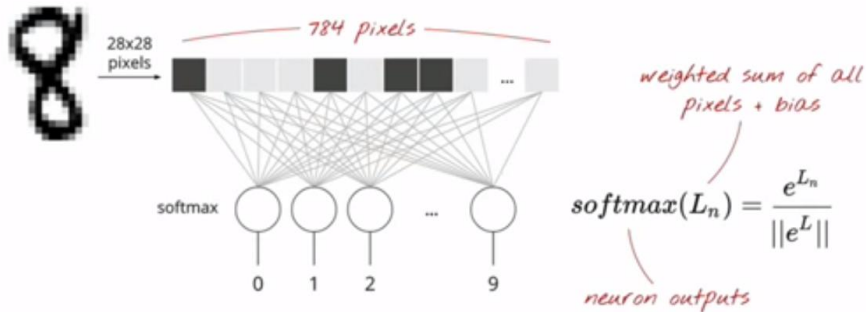


One-Hot Word Representations

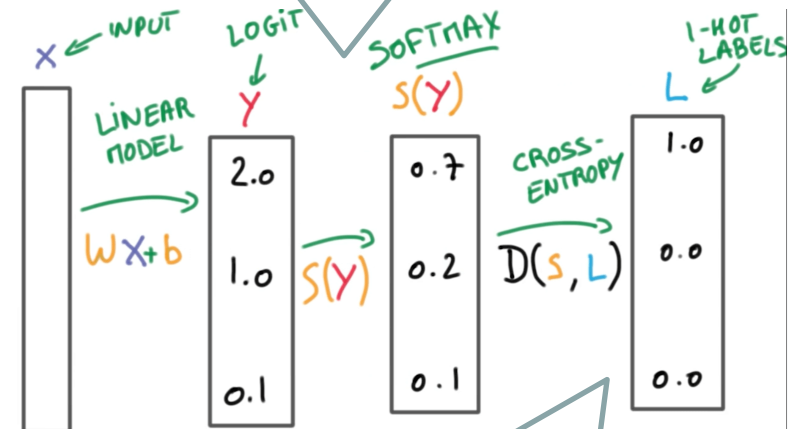
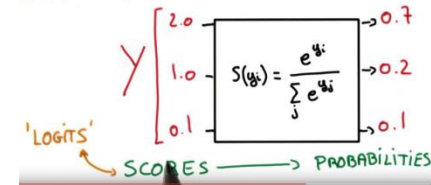


Softmax and Cross Entropy

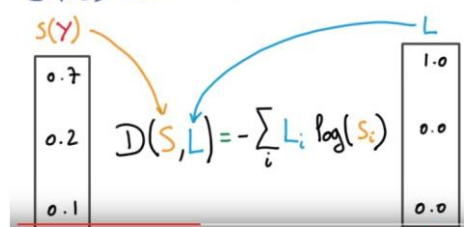
3 4 2 1 9 5 6 2 1 8
8 9 1 2 5 0 0 6 6 4
6 7 0 1 6 3 6 3 7 0
3 7 7 4 4 6 1 8 2
2 9 3 4 3 9 8 7 2 5
1 5 9 8 3 6 5 7 2 3
9 3 1 9 1 5 8 0 8 4
5 6 2 6 8 5 8 8 9 9
3 7 7 0 9 4 8 5 4 3
7 2 6 4 1 0 6 9 2 3



SOFTMAX



CROSS-ENTROPY



Predictions

$Y[100, 10]$

Images

$X[100, 784]$

Weights

$W[784, 10]$

Biases

$b[10]$

$$Y = \text{softmax}(X.W + b)$$

applied line
by line

matrix multiply

broadcast
on all lines

tensor shapes in []

<https://www.ritchieng.com/machine-learning/deep-learning/neural-nets/>
<https://www.youtube.com/watch?v=vq2nnJ4g6N0>
https://www.youtube.com/watch?v=tRsSi_sqXjl

Version 1



Linear Classifier

A single-layer neural network-based classification of handwritten digits form MNIST dataset

```
import tensorflow as tf
from numpy import argmax
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

# Import data
(train_images, train_labels), (test_images, test_labels) =
    tf.keras.datasets.mnist.load_data()

# Prepare output
unique_category_count = 10
print("Shape before one-hot encoding: ", train_labels.shape)
y_train = tf.one_hot(train_labels, unique_category_count)
# or tf.keras.utils.to_categorical()

y_test = tf.one_hot(test_labels, unique_category_count)
print("Shape after one-hot encoding: ", y_train.shape)

# Prepare input
train_images = train_images.astype('float32')
test_images = test_images.astype('float32')
print(train_images.shape)
train_images = tf.reshape(train_images, [60000, 784])
# or train_images.reshape(60000, 784)

test_images = tf.reshape(test_images, [10000, 784])
print(test_images.shape)

# normalizing the data to help with the training
train_images /= 255
test_images /= 255

# define model
model = Sequential()
model.add(Dense(10, activation='softmax'))
```

```
# compile the model
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
# fit the model
model.fit(train_images, y_train, epochs=2, batch_size=100, verbose=1)
# evaluate the model
loss, acc = model.evaluate(test_images, y_test, verbose=0)
print('Test Accuracy: %.3f % acc)
```

Test Accuracy: 0.914

```
# predict
k = 1
test_im = tf.reshape(test_images[k], [1, 784])
print(test_labels[k])
print(y_test[k].numpy())
yhat = model.predict(test_im)
print('Predicted: %s (class=%d)' % (yhat, argmax(yhat)))
```

2

[0. 0. 1. 0. 0. 0. 0. 0. 0. 0.]

Predicted: [[2.4658944e-03 8.4701322e-05 9.6051437e-01 7.8007090e-03 1.5989290e-08

1.1088169e-02 1.6074454e-02 2.4621694e-09 1.9715305e-03 2.2467559e-07]] (class=2)



Version 2



Linear Classifier

A single-layer neural network-based classification of handwritten digits form MNIST dataset

```
import tensorflow as tf
from numpy import argmax
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Flatten

# Import data
(train_images, train_labels), (test_images, test_labels) =
    tf.keras.datasets.mnist.load_data()

# Prepare input
train_images = train_images.astype('float32')
test_images = test_images.astype('float32')
print(train_images.shape)
# normalizing the data to help with the training
train_images /= 255
test_images /= 255

# define model
model = Sequential()
model.add(Flatten(input_shape=(28,28)))
model.add(Dense(10, activation='softmax'))
# compile the model
model.compile(optimizer='adam',
    loss='sparse_categorical_crossentropy', metrics=['accuracy'])
# fit the model
model.fit(train_images, train_labels, epochs=20, batch_size=100,
    verbose=1)
# evaluate the model
loss, acc = model.evaluate(test_images, test_labels, verbose=0)
print('Test Accuracy: %.3f' % acc)
```

Test Accuracy: 0.913 (with 2 epochs)
Test Accuracy: 0.924 (with 20 epochs)

```
# predict
k = 1
test_im = tf.reshape(test_images[k],[1,28,28])
print(test_labels[k])
yhat = model.predict(test_im)
print('Predicted: %s (class=%d)' % (yhat, argmax(yhat)))
```

```
2
Predicted: [[0.10497269 0.11022787 0.11731301 0.11628813 0.08157932 0.09450245
0.10957752 0.08391336 0.09888434 0.08274125]] (class=2)
```



Since TensorFlow 2.x includes Keras, there is no need to install it separately. Simply access it as:

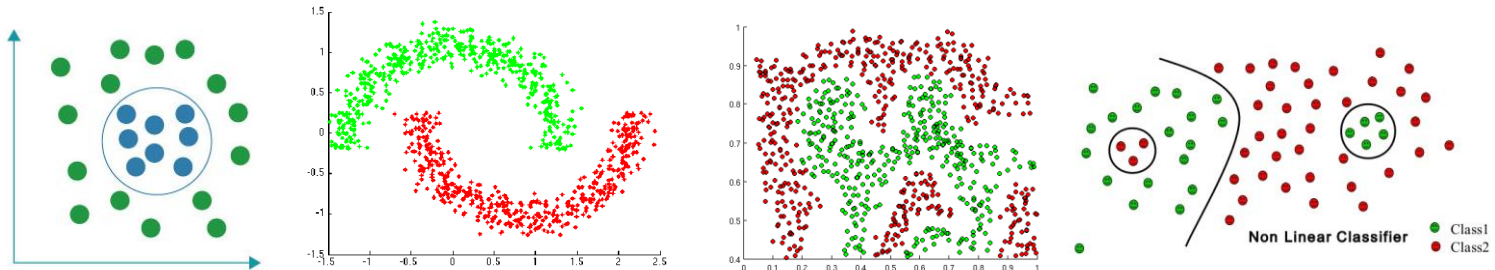
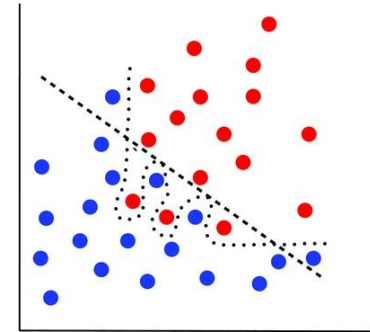
```
import tensorflow as tf
tf.keras. ...
```

Difference between 'categorical_crossentropy' and 'sparse_categorical_crossentropy' losses:

If use 'sparse_categorical_crossentropy', you do not need to transform the logist into categorical (via one hot representation).

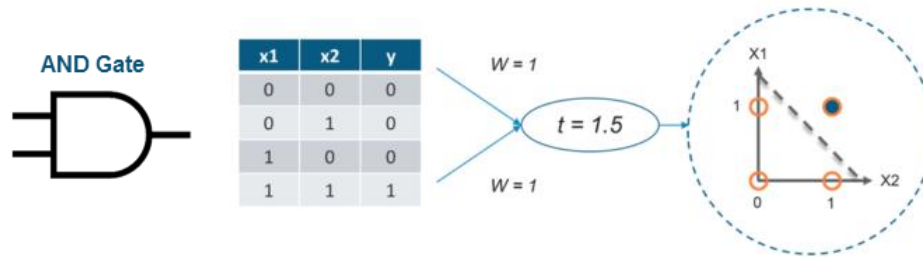
Classification

Linear model cannot serve non-linear classification problems...

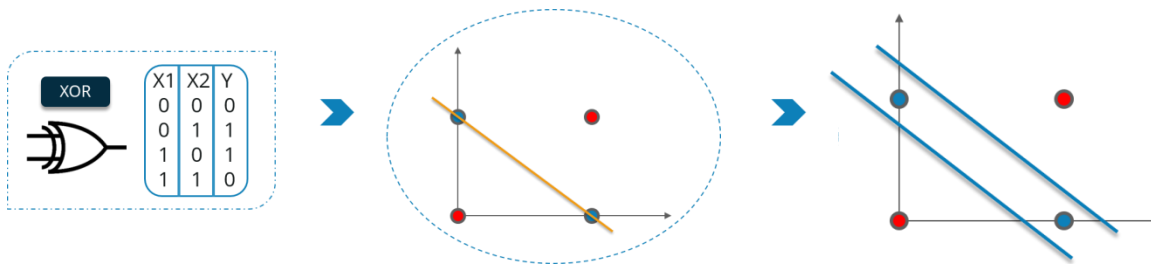


Let's go Deeper!!!

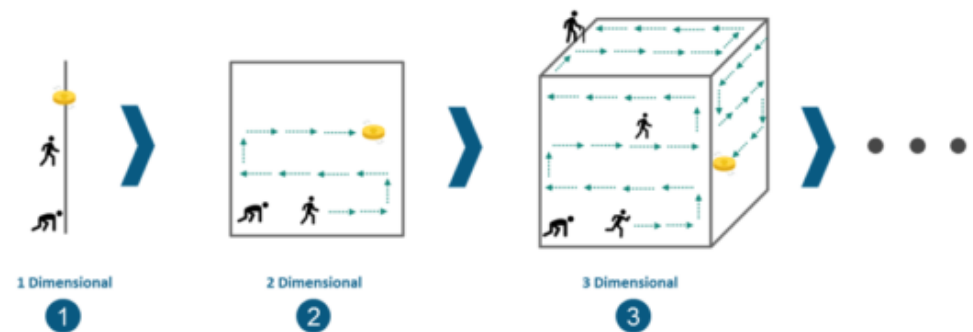
Neural Networks



High dimensionality is one of the bottlenecks for Machine Learning. Handling and processing high dimensional data (where input & output is quite large) becomes very complex and resource exhaustive.



Deep learning is capable of handling the high dimensional data being efficient in focusing on the right features on its own (feature extraction process).

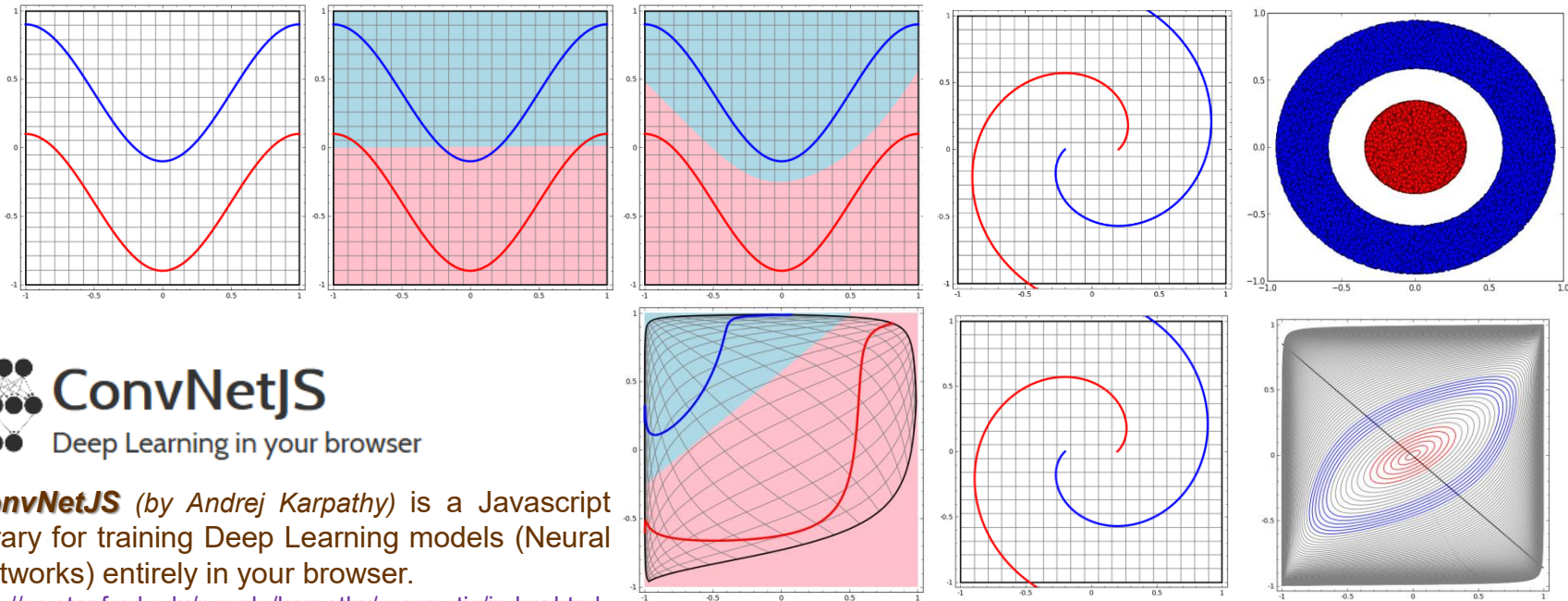
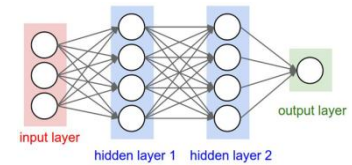


Relevant links:

<https://www.edureka.co/blog/deep-learning-tutorial#deep-learning-use-case>

Neural Networks

Neural Network change an original representation of data into the most appropriate representation to solve the problems ...



ConvNetJS (by Andrej Karpathy) is a Javascript library for training Deep Learning models (Neural Networks) entirely in your browser.

<http://cs.stanford.edu/people/karpathy/convnetjs/index.html>

Play with **ConvnetJS demo**: toy 2d classification with 2-layer neural network

<http://cs.stanford.edu/people/karpathy/convnetjs/demo/classify2d.html>

TensorFlow Playground - neural network right in your browser: <http://playground.tensorflow.org/>

<https://cloud.google.com/blog/products/ai-machine-learning/understanding-neural-networks-with-tensorflow-playground>

Relevant links:

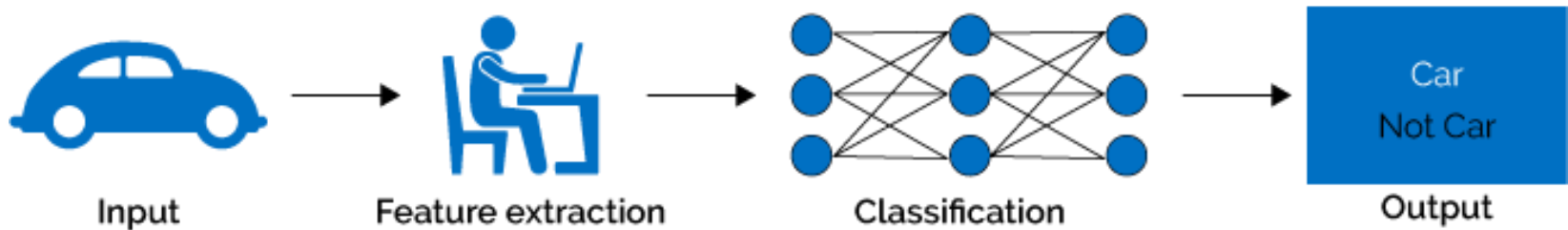
<http://colah.github.io/posts/2014-03-NN-Manifolds-Topology/>

<https://www.youtube.com/watch?v=BR9h47Jtqyw>

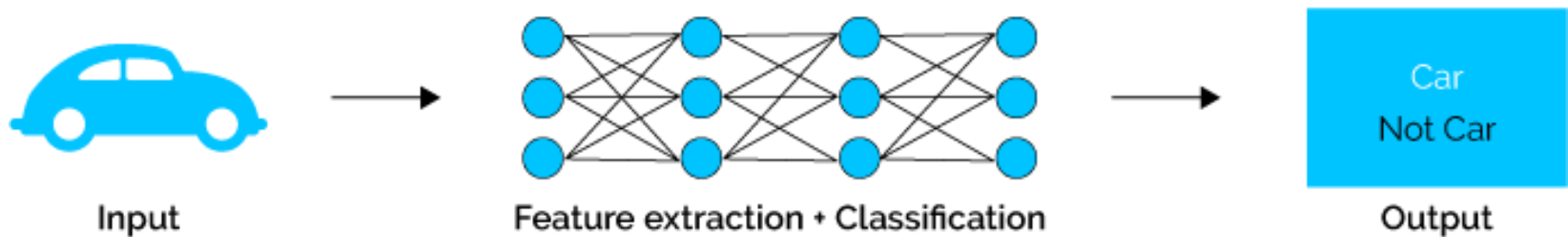
Neural Networks

One of the most exciting features of deep learning is its performance in **feature learning**; the algorithms perform particularly well in being able to detect features from raw data.

Machine Learning



Deep Learning



Relevant links:

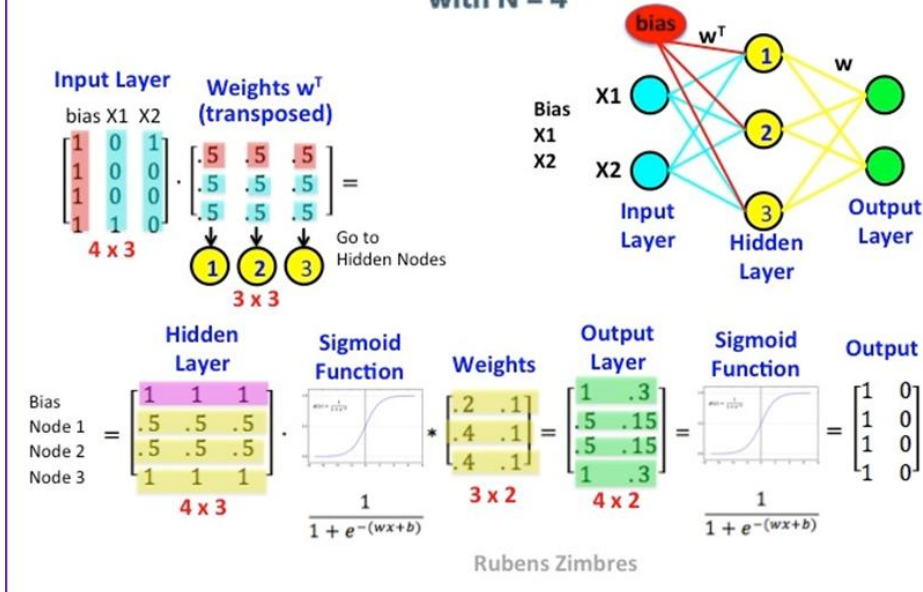
<https://udarajay.com/applied-machine-learning-the-less-confusing-guide/>

<https://www.youtube.com/watch?v=aircAruvnKk>

Neural Networks

Neural Networks

Color Guided Matrix Multiplication for a Binary Classification Task with $N = 4$



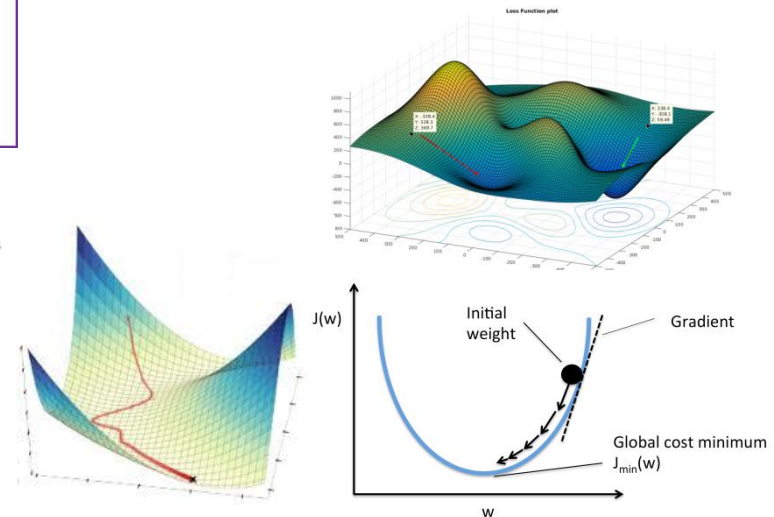
Forward propagation is a neural net's way of classifying a set of inputs (when input data goes through the nodes and their activation functions and finally form confidence levels for the nodes of output layer).

To train the net, the output from forward prop is compared to the output that is known to be correct, and the cost is the difference of the two. **The point of training is to make that cost as small as possible**, across millions of training examples. To do this, the net tweaks the weights and biases step by step until the prediction closely matches the correct output.

One way to train the model is called as **Backpropagation** - algorithm that looks for the minimum value of the error function in weight space using a technique called the delta rule or **gradient descent**. The weights that minimize the error function is then considered to be a solution to the learning problem.

Relevant links:

<https://www.edureka.co/blog/backpropagation/>
<https://en.wikipedia.org/wiki/Backpropagation>
<https://www.youtube.com/watch?v=IHZwWFHWa-w>
<https://www.youtube.com/watch?v=llg3gGewQ5U>



Non Linear Classification with DNN



Multi Layer Perceptron (MLP) Classification of handwritten digits form MNIST dataset

```
import tensorflow as tf
from numpy import argmax
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Flatten

# Import data
(train_images, train_labels), (test_images, test_labels) =
    tf.keras.datasets.mnist.load_data()

# Prepare input
train_images = train_images.astype('float32')
test_images = test_images.astype('float32')
print(train_images.shape)
# normalizing the data to help with the training
train_images /= 255
test_images /= 255

# define model
model = Sequential()
model.add(Flatten(input_shape=(28,28)))
model.add(Dense(256, activation='relu', kernel_initializer='he_normal'))
model.add(Dense(256, activation='relu', kernel_initializer='he_normal'))
model.add(Dense(10, activation='softmax'))
# compile the model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy', metrics=['accuracy'])
# fit the model
model.fit(train_images, train_labels, epochs=20, batch_size=100,
          verbose=1)
```

```
# evaluate the model
loss, acc = model.evaluate(test_images, test_labels, verbose=0)
print("Test Accuracy: %.3f % acc")
```

Test Accuracy: 0.980 (with 20 epochs)
Test Accuracy: 0.984 (with 40 epochs)

```
# predict
k = 1
test_im = tf.reshape(test_images[k],[1,28,28])
print(test_labels[k])
yhat = model.predict(test_im)
print("Predicted: %s (class=%d)" % (yhat, argmax(yhat)))
```

2

Predicted: [[1.2363079e-19 8.7495413e-25 1.0000000e+00 5.4430578e-20 9.0466738e-36
7.3684840e-29 4.6826463e-26 9.1150330e-33 1.9784944e-20 1.6744042e-37]] (class=2)



Iris flower dataset

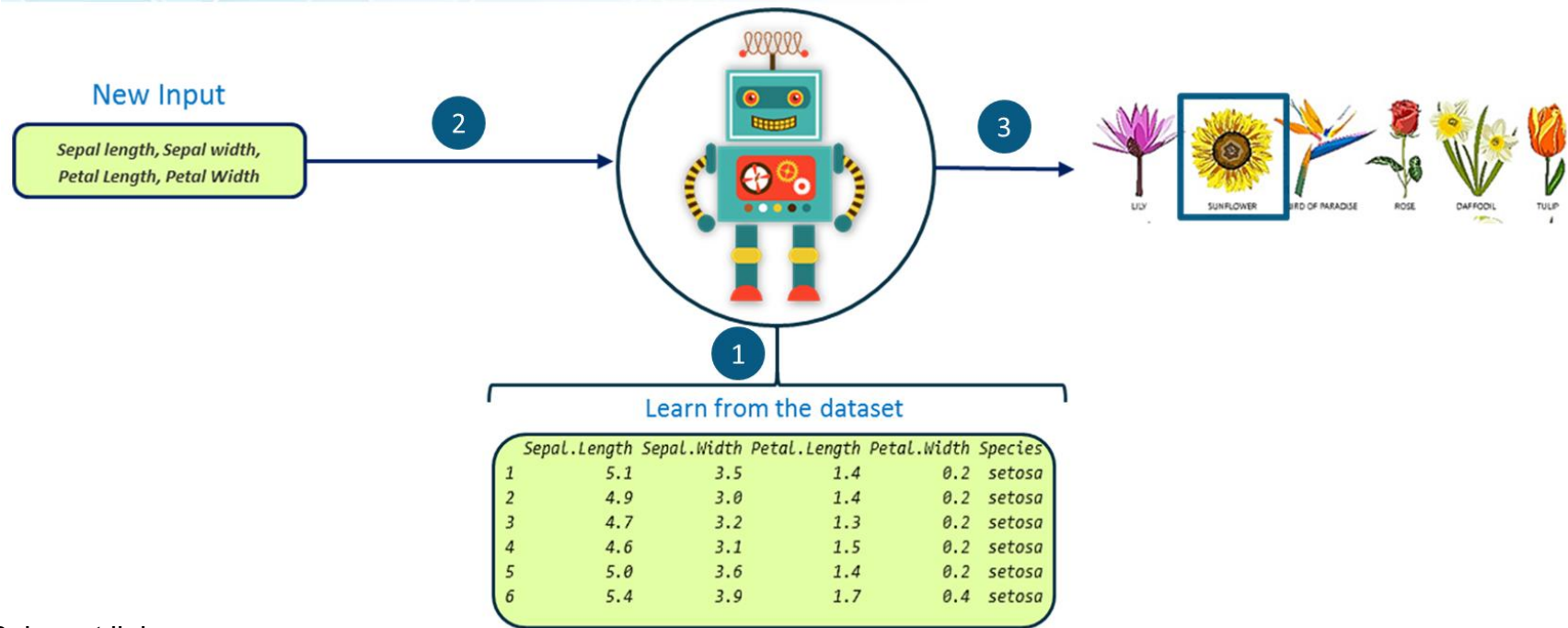
Iris flower dataset (Ronald Fisher's Iris data set): https://en.wikipedia.org/wiki/Iris_flower_data_set

The dataset contains a set of 50 records under 5 attributes - **Petal Length** , **Petal Width** , **Sepal Length** , **Sepal Width** and **Class**. The data set consists of 50 samples from each of three species of Iris (*Iris setosa*, *Iris virginica* and *Iris versicolor*).

<https://gist.github.com/curran/a08a1080b88344b0c8a7>

http://download.tensorflow.org/data/iris_training.csv

<https://raw.githubusercontent.com/jbrownlee/Datasets/master/iris.csv>



Relevant links:

<https://www.edureka.co/blog/deep-learning-tutorial#deep-learning-use-case>



Classification

```
import numpy as np
from numpy import argmax
from pandas import read_csv
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
# load the dataset
path =
'https://raw.githubusercontent.com/jbrownlee/Datasets/master/iris.csv'
df = read_csv(path, header=None)
# split into input and output columns
X, y = df.values[:, :-1], df.values[:, -1]
# ensure all data are floating point values and scale them
X = X.astype('float32')
scaler_X = MinMaxScaler()
scaler_X.fit(X)
X = scaler_X.transform(X)
# encode strings to integer
y = LabelEncoder().fit_transform(y)
# split into train and test datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
# determine the number of input features
n_features = X_train.shape[1]
# create dictionary of target classes
label_dict = {
    0: 'Setosa',
    1: 'Versicolor',
    2: 'Virginica'
}
```

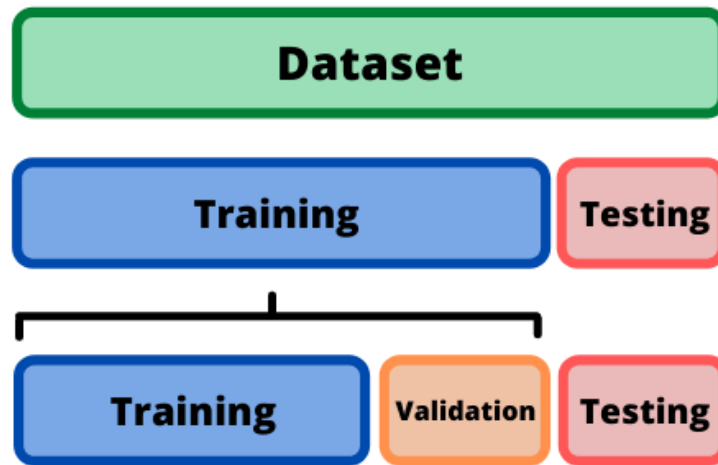
```
# define model
model = Sequential()
model.add(Dense(10, activation='relu', kernel_initializer='he_normal',
                input_shape=(n_features,)))
model.add(Dense(8, activation='relu', kernel_initializer='he_normal'))
model.add(Dense(3, activation='softmax'))
# compile the model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy', metrics=['accuracy'])
# fit the model
model.fit(X_train, y_train, epochs=500, batch_size=32, verbose=0)
# evaluate the model
loss, acc = model.evaluate(X_test, y_test, verbose=0)
print('Test Accuracy: %.3f' % acc)
```

Test Accuracy: 0.980

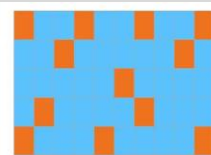
```
# Classify new flower samples.
new_samples = np.array([[6.4, 3.2, 4.5, 1.5],
                        [6.4, 2.8, 5.6, 2.2],
                        [7.0, 3.2, 4.7, 1.4],
                        [5.1, 3.3, 1.7, 0.5],
                        [5.8, 3.1, 5.0, 1.7]], dtype=np.float32)
new_samples = scaler_X.transform(new_samples)
yhat = model.predict(new_samples)
for i in range(len(yhat)):
    print('Class prediction {} : {}'.format(i+1,
                                             label_dict[int(argmax(yhat[i]))]))
```

Class prediction 1 : Versicolor
 Class prediction 2 : Virginica
 Class prediction 3 : Versicolor
 Class prediction 4 : Setosa
 Class prediction 5 : Versicolor

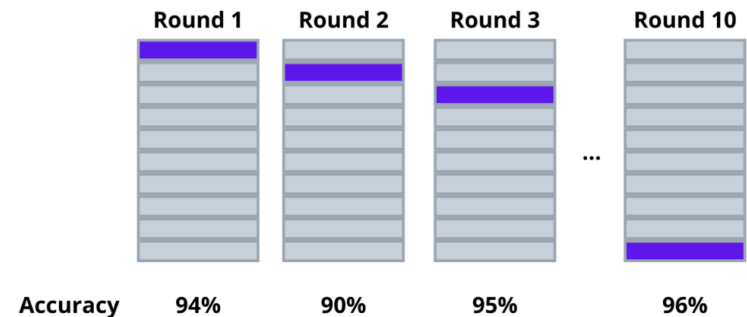
Training and Evaluation



```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split (
    X, y,
    test_size = 0.2,
    random_state = 1)
```



Validation Set
Training Set



Final Accuracy = Average(Round 1, Round 2, Round 3 ... Round 10)

... the error rate measured by **k-fold cross validation** might underestimate the true error rate once the model is applied to an unseen dataset...

The **validation set** is used to optimize the model parameters while the **test set** is used to provide an unbiased estimate of the final model...

Relevant links:

<https://sdsclub.com/how-to-train-and-test-data-like-a-pro/>

<https://towardsdatascience.com/addressing-the-difference-between-keras-validation-split-and-sklearn-s-train-test-split-a3fb803b733>

<https://machinelearningmastery.com/evaluate-performance-deep-learning-models-keras/>

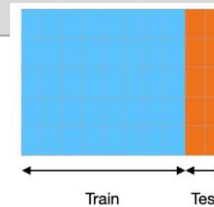
https://www.tensorflow.org/guide/keras/train_and_evaluate

Training and Evaluation

Fit the model...

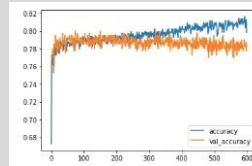
```
# fit model on training data and pass some validation for monitoring validation loss and metrics  
# at the end of each epoch  
history = model.fit(x_train, y_train, epochs=2, validation_data=(x_val, y_val))
```

```
history = model.fit( ... , validation_split=0.2)
```



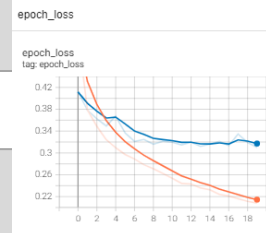
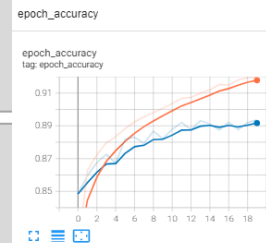
Visualize the training process...

```
import matplotlib.pyplot as plt  
# plot the model accuracy and validation accuracy  
plt.plot(history.history['accuracy'])  
plt.plot(history.history['val_accuracy'])  
plt.legend(['accuracy', 'validation accuracy'])
```



```
%load_ext tensorboard
```

```
...  
tensorboard_callback = tf.keras.callbacks.TensorBoard(log_dir="./logs")  
model.fit(x_train, y_train, epochs=2, callbacks=[tensorboard_callback])  
# run the tensorboard command to view the visualizations.  
%tensorboard --logdir './logs'
```



Evaluate and predict...

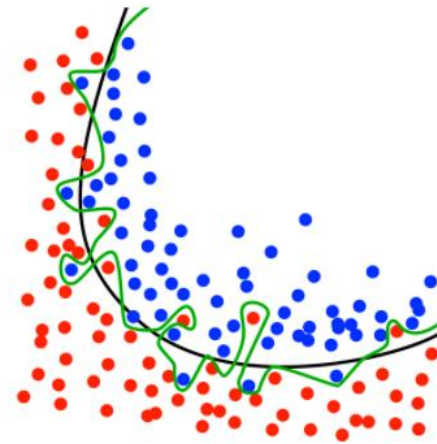
```
# Evaluate the model on the test data using `evaluate`  
results = model.evaluate(x_test, y_test)  
# Generate predictions  
predictions = model.predict(x_test)
```

fit method

```
Model.fit(  
    x=None,  
    y=None,  
    batch_size=None,  
    epochs=1,  
    verbose="auto",  
    callbacks=None,  
    validation_split=0.0,  
    validation_data=None,  
    shuffle=True,  
    class_weight=None,  
    sample_weight=None,  
    initial_epoch=0,  
    steps_per_epoch=None,  
    validation_steps=None,  
    validation_batch_size=None,  
    validation_freq=1,  
    max_queue_size=10,  
    workers=1,  
    use_multiprocessing=False,  
)
```

Overfitting

Overfitting is "the production of an analysis that corresponds too closely or exactly to a particular set of data, and may therefore fail to fit additional data or predict future observations reliably"



Classification with overfitting (green) vs. better classification boundary (black)

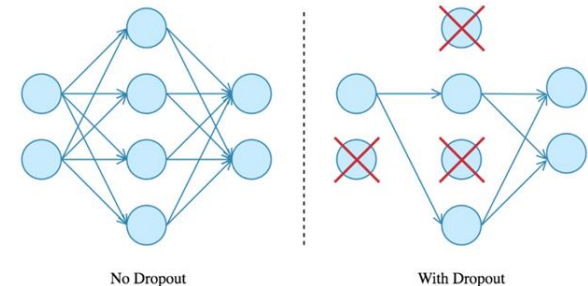
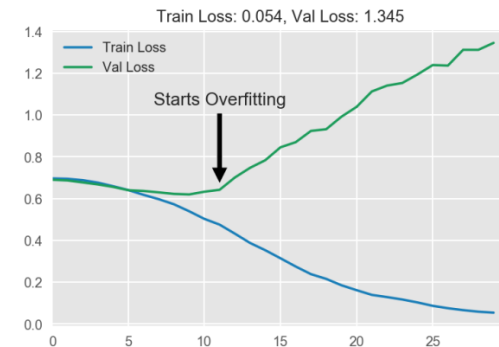
We may face a case, when training loss keeps going down, but the validation loss starts increasing after certain epoch. It is a sign of **overfitting**. It tells us that our model is memorizing the training data, but it's failing to generalize to new instances.

The most popular regularization technique for deep neural networks is **Dropout**. It is used to prevent overfitting via temporarily "dropping"/disabling a neuron with probability p (a hyperparameter called the **dropout-rate** that is typically a number around 0.5) at each iteration during the training phase.

- The dropped-out neurons are resampled with probability p at every training step (a dropped out neuron at one step can be active at the next one). (Dropout is not applied during test time after the network is trained).
- Dropout can be applied to input or hidden layer nodes, but not the output nodes.
- Applying dropout regularization, it is recommended to increase a number of training rounds.

Reason. Dropout forces every neuron to be able to operate independently and prevents the network to be too dependent on a small number of neurons.

Relevant links:





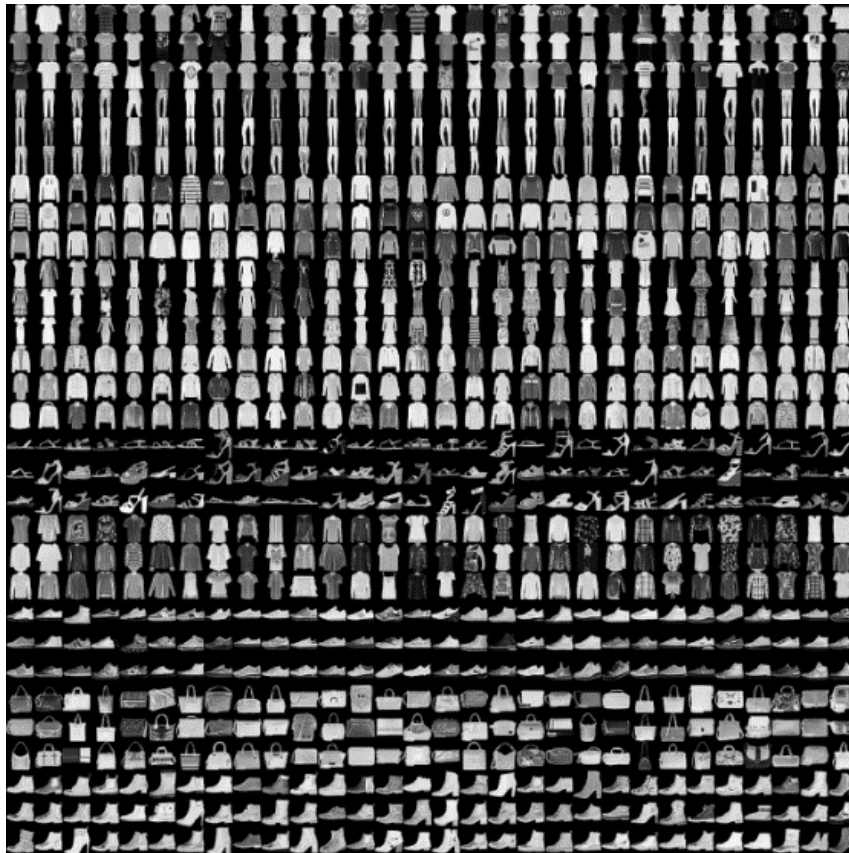
Classification

Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. Han Xiao, Kashif Rasul, Roland Vollgraf

<https://arxiv.org/abs/1708.07747>

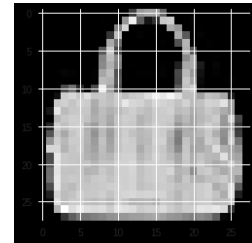
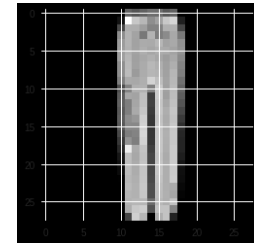
<https://github.com/zalando-research/fashion-mnist>

<https://medium.com/tensorist/classifying-fashion-articles-using-tensorflow-fashion-mnist-f22e8a04728a>



Label Description

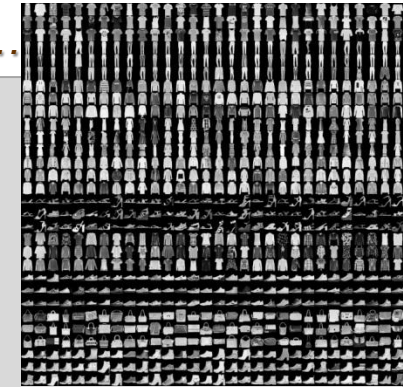
- 0 T-shirt
- 1 Trouser
- 2 Pullover
- 3 Dress
- 4 Coat
- 5 Sandal
- 6 Shirt
- 7 Sneaker
- 8 Bag
- 9 Ankle boot





Classification

Different model performance comparison with Fashion-MNIST dataset...



```
import tensorflow as tf
from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.models import Sequential, load_model
from tensorflow.keras.layers import Dense, Dropout, Flatten
from tensorflow.keras.optimizers import RMSprop
from tensorflow.keras.optimizers import SGD
import os
from numpy import argmax
import matplotlib.pyplot as plt
%matplotlib inline

batch_size = 128
num_classes = 10
epochs = 20
# the data, shuffled and split between train and test sets
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
# check some data
print(y_train[1000])
fig = plt.figure()
plt.imshow(x_train[1000], cmap='gray')
fig = plt.figure(figsize=(20,10))
for i in range(1,41):
    ax1 = fig.add_subplot(5,8,i)
    plt.xticks([], [])
    plt.yticks([], [])
    ax1.imshow(x_train[i], cmap='gray')
# normalizing the data to help with the training
x_train = x_train.astype('float32')
x_test = x_test.astype('float32')
x_train /= 255
x_test /= 255
```

```
# prepare output dictionary
```

```
di = {0:'T-shirt',
      1:'Trouser',
      2:'Pullover',
      3:'Dress',
      4:'Coat',
      5:'Sandal',
      6:'Shirt',
      7:'Sneaker',
      8:'Bag',
      9:'Ankle boot'}
```

```
def train_model(model):
```

```
    history = model.fit(x_train, y_train, batch_size=batch_size, epochs=epochs,
                        verbose=1, validation_data=(x_test, y_test))
```

```
    score = model.evaluate(x_test, y_test, verbose=0)
```

```
    print('Test loss:', score[0])
```

```
    print('Test accuracy:', score[1])
```

```
    return model
```

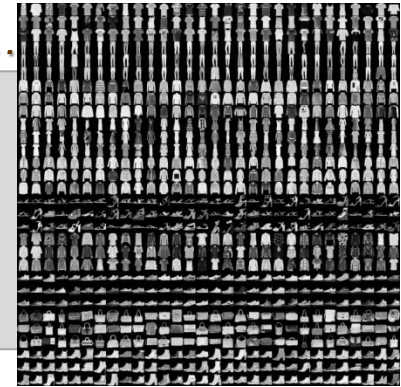
```
# define and train the model
```

```
...
```



Classification

Different model performance comparison with Fashion-MNIST dataset...



```
# Single layer with 1 unit
model0 = Sequential()
model0.add(Flatten(input_shape=(28,28)))
model0.add(Dense(1, activation='relu', kernel_initializer='he_normal'))
model0.add(Dense(num_classes, activation='softmax'))
model0.summary()
model0.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
model_ = train_model(model0)
```

```
...
Epoch 20/20
469/469 [=====] - 0s 719us/step - loss: 1.4434 - accuracy: 0.4016 - val_loss: 1.4556 - val_accuracy: 0.3984
Test loss: 1.4555656909942627
Test accuracy: 0.3984000086784363
```

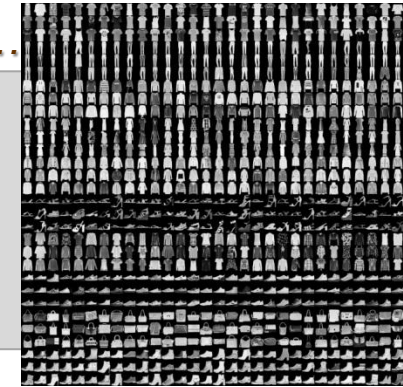
```
# Single layer with 10 units
model1 = Sequential()
model1.add(Flatten(input_shape=(28,28)))
model1.add(Dense(10, activation='relu', kernel_initializer='he_normal'))
model1.add(Dense(num_classes, activation='softmax'))
model1.summary()
model1.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
model_ = train_model(model1)
```

```
...
Epoch 20/20
469/469 [=====] - 0s 860us/step - loss: 0.3749 - accuracy: 0.8705 - val_loss: 0.4374 - val_accuracy: 0.8457
Test loss: 0.437436580657959
Test accuracy: 0.8457000255584717
```



Classification

Different model performance comparison with Fashion-MNIST dataset...



```
# Single Layer with more units
model2 = Sequential()
model2.add(Flatten(input_shape=(28,28)))
model2.add(Dense(512, activation='relu', kernel_initializer='he_normal'))
model2.add(Dense(num_classes, activation='softmax'))
model2.summary()
model2.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
model_ = train_model(model2)
```

```
...
Epoch 20/20
469/469 [=====] - 1s 3ms/step - loss: 0.1522 - accuracy: 0.9437 - val_loss: 0.3311 - val_accuracy: 0.8922
Test loss: 0.3311215341091156
Test accuracy: 0.8921999931335449
```

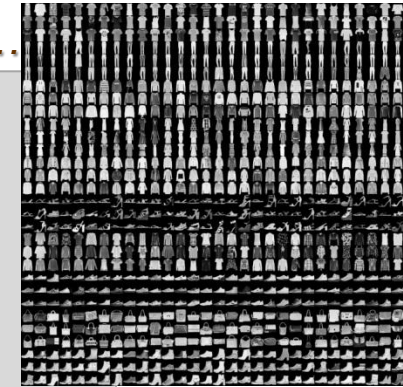
```
# Multiple Layers Without dropout
model3 = Sequential()
model3.add(Flatten(input_shape=(28,28)))
model3.add(Dense(512, activation='relu', kernel_initializer='he_normal'))
model3.add(Dense(512, activation='relu', kernel_initializer='he_normal'))
model3.add(Dense(num_classes, activation='softmax'))
model3.summary()
model3.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
model_ = train_model(model3)
```

```
...
Epoch 20/20
469/469 [=====] - 3s 7ms/step - loss: 0.1354 - accuracy: 0.9472 - val_loss: 0.3703 - val_accuracy: 0.8910
Test loss: 0.37025687098503113
Test accuracy: 0.890999972820282
```



Classification

Different model performance comparison with Fashion-MNIST dataset...



```
# Multiple Layers With dropout
model4 = Sequential()
model4.add(Flatten(input_shape=(28,28)))
model4.add(Dense(512, activation='relu', kernel_initializer='he_normal'))
model4.add(Dropout(0.2))
model4.add(Dense(512, activation='relu', kernel_initializer='he_normal'))
model4.add(Dropout(0.2))
model4.add(Dense(num_classes, activation='softmax'))
model4.summary()
model4.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
model_ = train_model(model4)
```

```
...
Epoch 20/20
469/469 [=====] - 4s 8ms/step - loss: 0.2071 - accuracy: 0.9214 - val_loss: 0.3317 - val_accuracy: 0.8921
Test loss: 0.3317197263240814
Test accuracy: 0.8920999765396118
```

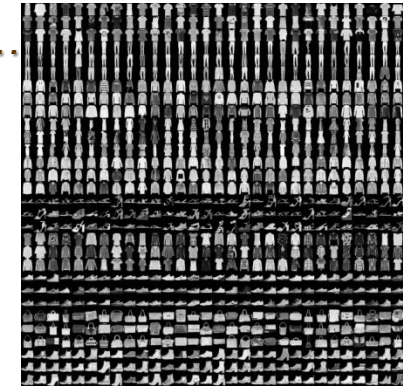
```
# Multiple Layers With dropout
model5 = Sequential()
model5.add(Flatten(input_shape=(28,28)))
model5.add(Dense(512, activation='relu', kernel_initializer='he_normal'))
model5.add(Dropout(0.2))
model5.add(Dense(512, activation='relu', kernel_initializer='he_normal'))
model5.add(Dropout(0.2))
model5.add(Dense(num_classes, activation='softmax'))
model5.summary()
model5.compile(loss='sparse_categorical_crossentropy', optimizer=RMSprop(), metrics=['accuracy'])
model_ = train_model(model5)
```

```
...
Epoch 20/20
469/469 [=====] - 5s 11ms/step - loss: 0.2657 - accuracy: 0.9084 - val_loss: 0.3876 - val_accuracy: 0.8909
Test loss: 0.38759610056877136
Test accuracy: 0.8909000158309937
```



Classification

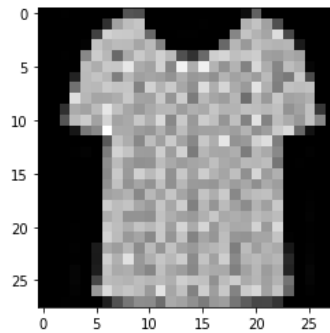
Different model performance comparison with Fashion-MNIST dataset...



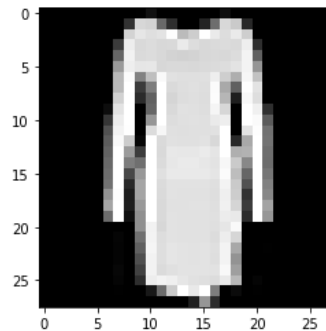
```
# saving the model
save_dir = "results/"
model_name = 'fashion_mnist.keras'
model_path = os.path.join(save_dir, model_name)
model_.save(model_path)
print('Saved trained model at %s ' % model_path)

#evaluation and prediction
model_ = load_model('results/fashion_mnist.keras')
loss_and_metrics = model_.evaluate(x_test, y_test, verbose=2)
print("Test Loss", loss_and_metrics[0])
print("Test Accuracy", loss_and_metrics[1])
```

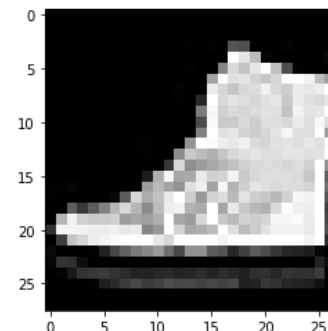
is classified as: Shirt



is classified as: Dress



is classified as: Ankle boot



Relevant links:

Save a Keras model: https://www.tensorflow.org/tutorials/keras/save_and_load

<https://medium.com/@juanc.olamendy/persisting-and-loading-keras-models-a-comprehensive-guide-25b9decf7140>



Lecture 3: AutoEncoders and Convolutional Neural Networks (CNN)

TIES4911 Deep-Learning for Cognitive Computing for Developers
Spring 2026

by:
Dr. Oleksiy Khriyenko
IT Faculty
University of Jyväskylä

UNLABELED data

- ❑ Feature Extraction
- ❑ Unsupervised Learning
- ❑ Pattern Recognition

use **Unsupervised Learning** (extraction of the patterns from a set of unlabeled data):

- Restricted Boltzmann Machine (RBM)
- Autoencoders

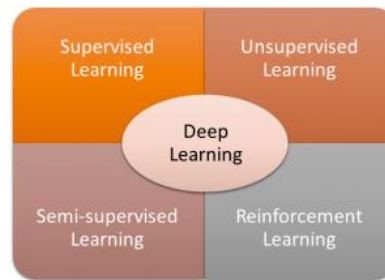
LABELED data

- ❑ Supervised Learning

use **Supervised Learning** to build predictors, classifiers, generators, etc. Depending on application use:

- Multilayer Perceptrons and Deep Belief Networks for regression and classification tasks.
- Convolutional Net (CNN) and Transformers for image and video processing (classification, annotation, generation, etc.)
- CNN for object detection/recognition.
- Recurrent Neural Net (RNN) and Transformers for sequences or time series analysis including speech recognition/generation and language model-based NLP tasks such as sentiment analysis, parsing, named entity recognition, etc.

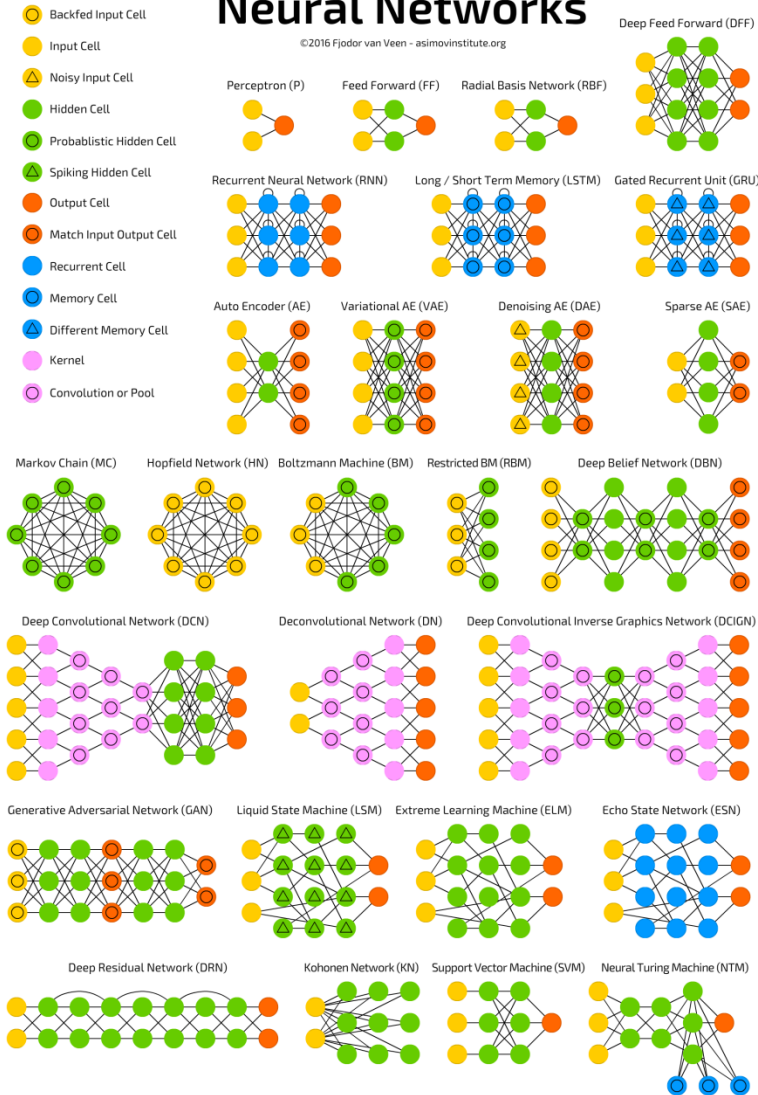
A visual and intuitive understanding of deep learning:
https://www.youtube.com/watch?v=Oqm9vsf_hvU



Deep Learning

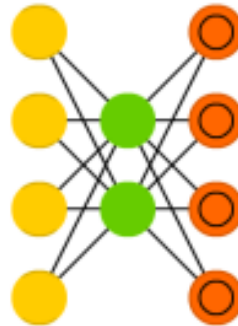
A mostly complete chart of Neural Networks

©2016 Fjodor van Veen - asimovinstitute.org



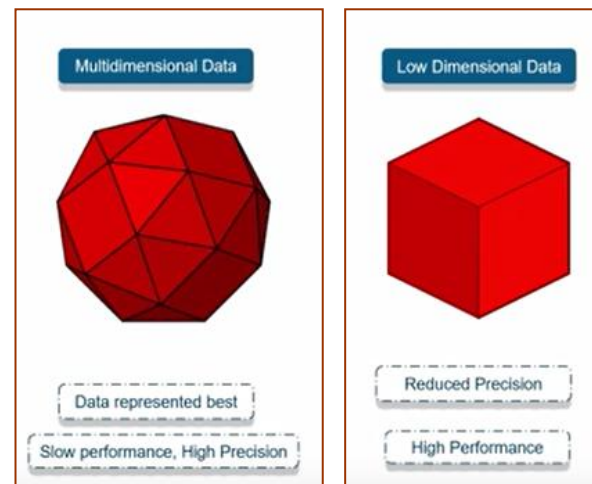
Autoencoders

Auto Encoder (AE)



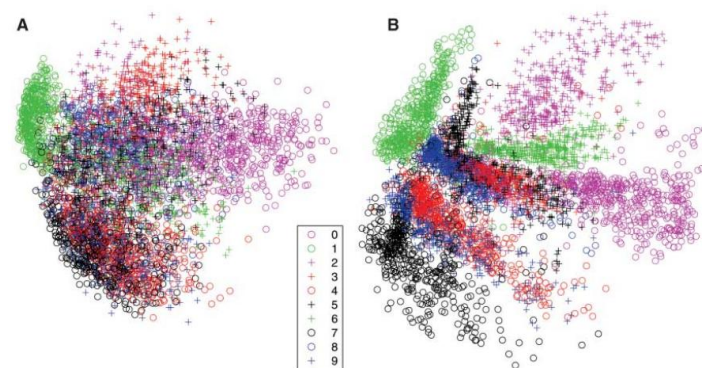
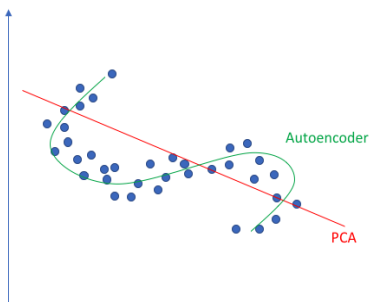
Autoencoders in Deep Learning

Data compression and **Dimensionality reduction** help us to increase the performance by converting our data into a smaller representation that we can recreate to a degree of quality.



Autoencoders outperform linear dimensionality reduction approaches (e.g. **Principle Component Analysis (PCA)**) applying non-linear transformations using multiple layers and non-linear activation functions...

Linear vs nonlinear dimensionality reduction



Relevant links:

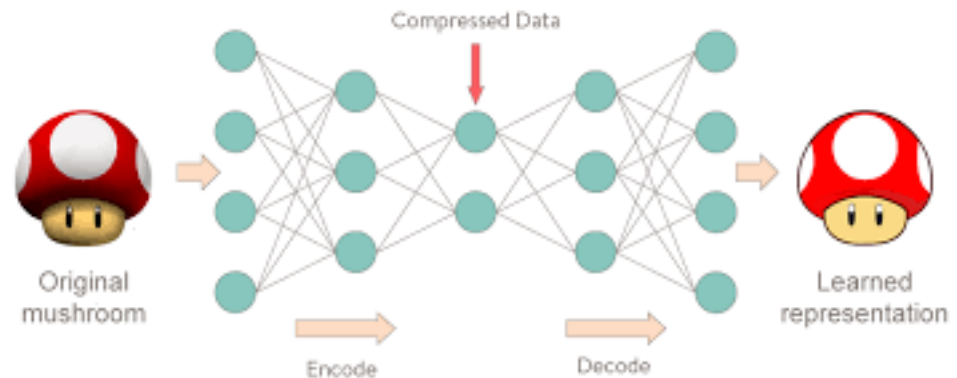
https://www.youtube.com/watch?v=nTt_ajul8NY

<http://www.deeplearningbook.org/contents/autoencoders.html>

<https://stats.stackexchange.com/questions/190148/building-an-autoencoder-in-tensorflow-to-surpass-pca>

Autoencoders in Deep Learning

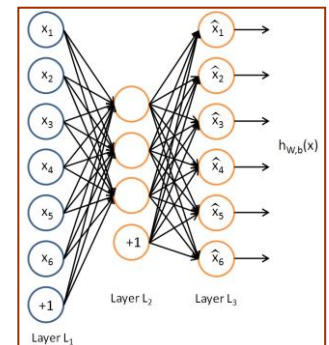
Autoencoders are an important family of neural networks that are designed to recognize inherent patterns in data and sort of encode their own structure. Autoencoder is a neural net that takes a set of typically unlabeled inputs, and after encoding them, tries to reconstruct them as accurately as possible. As a result, the net decides which of the data features are the most important, essentially acting as a feature extraction engine.



Autoencoders are typically very shallow, and are usually comprised of an input layer, an output layer and a hidden layer. However, there are deep autoencoders that are extremely useful tools for dimensionality reduction.

Autoencoder training parameters:

- **Code size** – size of compressed data (number of nodes in the middle layer)
- **Number of Layers** – could be as deep as you wish
- **Loss Function** – mean square error or binary cross entropy (if the input values in the range $[0, 1]$)
- **Number of Nodes per Layers** – usually layers structure is symmetric for encoder and decoder, and number of nodes decrease the “code” (compressed data)

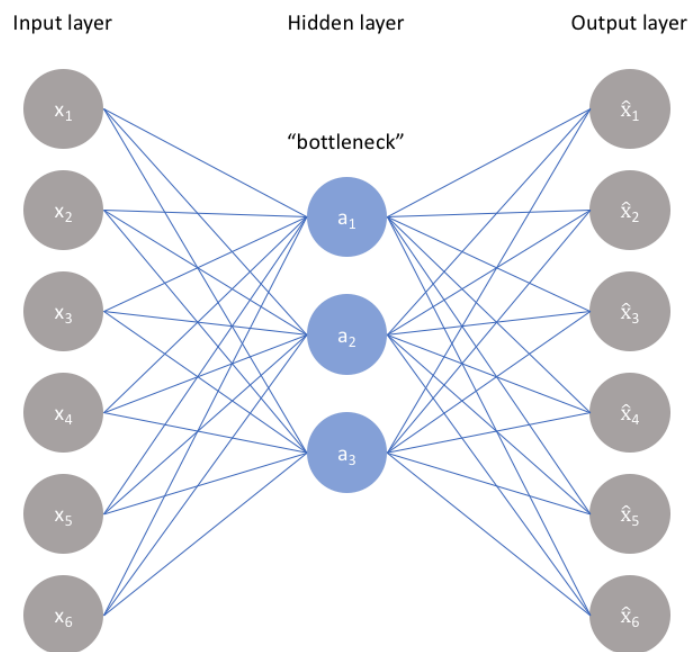


Relevant links:

https://www.youtube.com/watch?v=nTt_ajul8NY

<http://www.deeplearningbook.org/contents/autoencoders.html>

Autoencoders in Deep Learning



Loss function (usually consists of two terms: one encourage the model to be sensitive to the inputs (**reconstruction loss**), another one discourages memorization/overfitting (added **regularizer**).

$$\mathcal{L}(x, \hat{x}) + \text{regularizer}$$

For **binary inputs** calculate distance as cross-entropy (sum of Bernoulli cross-entropies):

$$l(f(\mathbf{x})) = - \sum_k (x_k \log(\hat{x}_k) + (1 - x_k) \log(1 - \hat{x}_k))$$

For **real-values inputs** use a linear activation function at the output and calculate distance as sum of squared differences (squared Euclidian distance):

$$l(f(\mathbf{x})) = \frac{1}{2} \sum_k (\hat{x}_k - x_k)^2$$

Relevant links:

<https://www.jeremyjordan.me/autoencoders/>

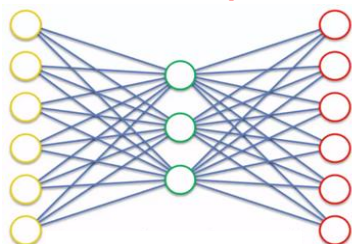
<https://www.youtube.com/watch?v=xTU79Zs4XKY>

<https://www.youtube.com/watch?v=EehRcPo1M-Q>

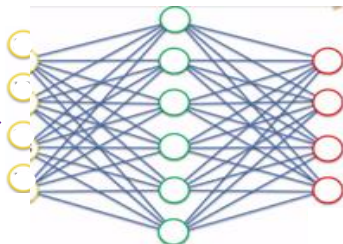
<https://keras.io/api/layers/regularizers/>

Autoencoders in Deep Learning

Undercomplete



Overcomplete



<vs>

Overcomplete Hidden Layers – a concept in which autoencoders may recognize more features, having a hidden layer that is equal to the number of inputs nodes or greater. However, it might bring a huge problem when trained and make it useless...

Both types of autoencoders can be shallow or deep...

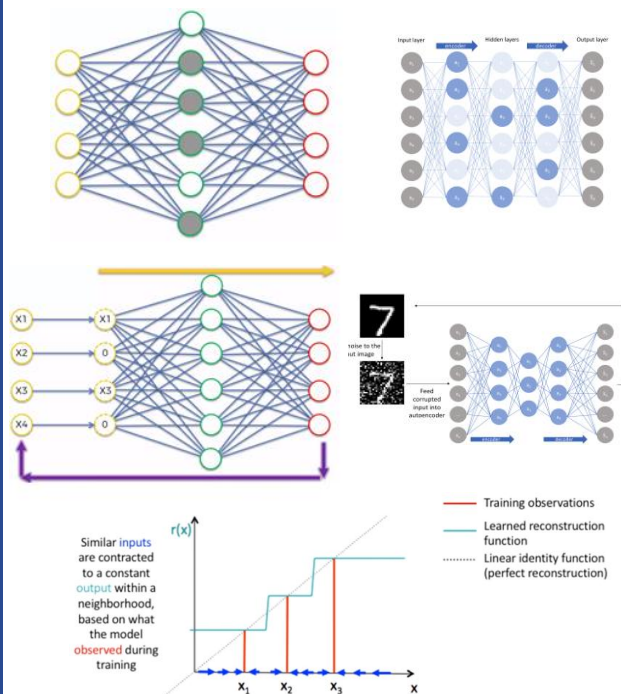
Sparse autoencoders – is an approach towards regularization via regularization of the activations, not the weights of a network. They offer an alternative method for introducing an information bottleneck without requiring a reduction in the number of nodes at the hidden layers. Here loss function supposed to penalize activations within a layer.

Denoising autoencoders – is an approach towards developing a generalizable model by slightly corrupting the input data but still maintaining the uncorrupted data as a target output.

Contractive autoencoders add a penalty to the backpropagation of the autoencoder preventing it from cheating. Here we explicitly encouraging the model to learn an encoding in which similar inputs have similar encodings.

Denoising autoencoders make the reconstruction function (ie. decoder) resist small but finite-sized perturbations of the input, while **contractive autoencoders** make the feature extraction function (ie. encoder) resist infinitesimal perturbations of the input. Jacobian of encoder added to loss function prevent the encoder from copying the input...

Autoencoders learn how to compress the data based on attributes (ie. correlations between the input feature vector) discovered from data during training, these models are typically **only capable of reconstructing data similar to the class of observations of which the model observed during training.**



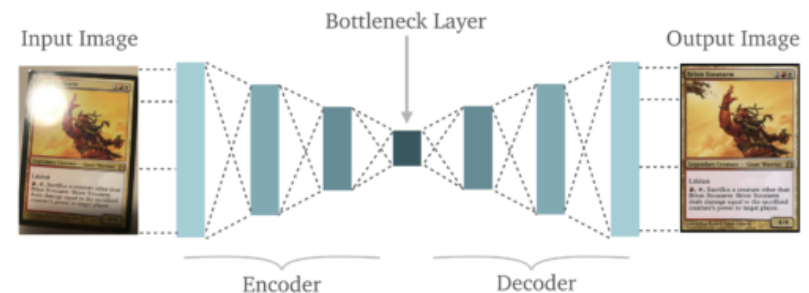
Relevant links:

<https://www.jeremyjordan.me/autoencoders/>

<https://medium.com/aimonks/contractive-autoencoders-an-insight-into-enhanced-feature-learning-d3d3bd103d88>

Autoencoders Use-cases

- **Data compression and dimensionality reduction**
- **Image reconstruction**
- **Image coloring**
- **Feature variation and Denoising**
- **Generating higher resolution images**
- **Image search and information retrieval**
- **Anomaly detection**
- ...



Relevant links:

<https://www.edureka.co/blog/autoencoders-tutorial/>

<https://medium.com/@piyushkashyap045/a-comprehensive-guide-to-autoencoders-8b18b58c2ea6>



Autoencoders

Reconstruction on MNIST dataset...

```
import matplotlib.pyplot as plt
import numpy as np
import tensorflow as tf
from sklearn.model_selection import train_test_split
from tensorflow.keras import layers, losses
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Model

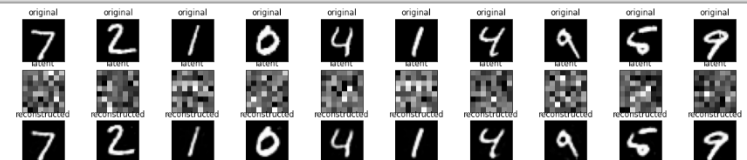
# size of encoded representations
latent_dim = 64 # compression (vs. the input is 784)
# define autoencoder class
class Autoencoder(Model):
    def __init__(self, latent_dim):
        super(Autoencoder, self).__init__()
        self.latent_dim = latent_dim
        # model that maps an input to its latent/encoded representation
        self.encoder = tf.keras.Sequential([
            layers.Flatten(),
            layers.Dense(latent_dim, activation='relu'),
        ])
        # model that maps an encoded representation to reconstructed
        self.decoder = tf.keras.Sequential([
            layers.Dense(784, activation='sigmoid'),
            layers.Reshape((28, 28))
        ])
    def call(self, x):
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return decoded

# model that maps an input to its reconstruction
autoencoder = Autoencoder(latent_dim)
# configure the model
autoencoder.compile(optimizer='adam',
                    loss=losses.MeanSquaredError())
```



```
# read dataset
(x_train, _), (x_test, _) = mnist.load_data()
# normalize all values between 0 and 1
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.
# fit autoencoder
autoencoder.fit(x_train, x_train, epochs=10, shuffle=True,
                validation_data=(x_test, x_test))

# encode and decode some digits, taking them from the *test* set
encoded_imgs = autoencoder.encoder(x_test).numpy()
decoded_imgs = autoencoder.decoder(encoded_imgs).numpy()
# visualise the results
n = 10 # how many digits we will display
plt.figure(figsize=(20, 4))
for i in range(n):
    # display original
    ax = plt.subplot(3, n, i + 1)
    plt.imshow(x_test[i])
    plt.gray()
    # display latent code
    ax = plt.subplot(3, n, i + 1 + n)
    plt.imshow(encoded_imgs[i].reshape(8, 8))
    plt.gray()
    # display reconstruction
    ax = plt.subplot(3, n, i + 1 + 2*n)
    plt.imshow(decoded_imgs[i])
    plt.gray()
plt.show()
```



Relevant links:

<https://www.tensorflow.org/tutorials/generative/autoencoder>



Autoencoders

Reconstruction on MNIST dataset...

```
import numpy as np
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model
from tensorflow.keras.datasets import mnist
import matplotlib.pyplot as plt

# size of encoded representations
encoding_dim = 32 # compression (vs. the input is 784)
# input placeholder
input_img = Input(shape=(784,))
# encoded representation of the input
encoded = Dense(encoding_dim, activation='relu')(input_img)
# lossy reconstruction of the input
decoded = Dense(784, activation='sigmoid')(encoded)
# model that maps an input to its reconstruction
autoencoder = Model(input_img, decoded)
# model that maps an input to its encoded representation
encoder = Model(input_img, encoded)
# create a placeholder for an encoded (32-dimensional) input
encoded_input = Input(shape=(encoding_dim,))
# retrieve the last layer of the autoencoder model
decoder_layer = autoencoder.layers[-1]
# decoder model
decoder = Model(encoded_input, decoder_layer(encoded_input))
# configure the model
autoencoder.compile(optimizer='adam', loss='mse')
# read dataset
(x_train, _), (x_test, _) = mnist.load_data()
# normalize all values between 0 and 1 and flatten
# the 28x28 images into vectors of size 784.
```



```
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.
x_train = x_train.reshape((len(x_train), np.prod(x_train.shape[1:])))
x_test = x_test.reshape((len(x_test), np.prod(x_test.shape[1:])))

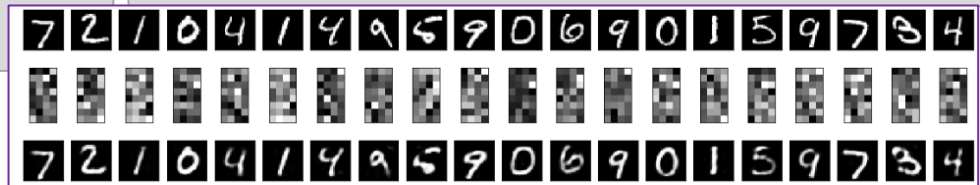
#fit autoencoder
autoencoder.fit(x_train, x_train, epochs=50, batch_size=256,
               shuffle=True, validation_data=(x_test, x_test))

# encode and decode some digits, taking them from the *test* set
encoded_imgs = encoder.predict(x_test)
decoded_imgs = decoder.predict(encoded_imgs)

# visualise the results
n = 20 # how many digits we will display
plt.figure(figsize=(20, 4))
for i in range(n):
    # display original
    ax = plt.subplot(3, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28))
    plt.gray()

    # display code
    ax = plt.subplot(3, n, i + 1 + n)
    plt.imshow(encoded_imgs[i].reshape(8, 4))
    plt.gray()

    # display reconstruction
    ax = plt.subplot(3, n, i + 1 + 2*n)
    plt.imshow(decoded_imgs[i].reshape(28, 28))
    plt.gray()
plt.show()
```



Relevant links:

<https://blog.keras.io/building-autoencoders-in-keras.html>



Autoencoders

Different architectures of the models...

```
### Vanila Net
def getVanilaAE():
    # input layer
    input_layer = Input(shape=(784,))
    # latent view
    latent_view = Dense(64, activation='relu')(input_layer)
    # output layer
    output_layer = Dense(784, activation='sigmoid')(latent_view)
    # model
    model = Model(input_layer, output_layer)

    return model
```

```
### Deep Net
def getDeepAE():
    # input layer
    input_layer = Input(shape=(784,))
    # encoding architecture
    encode_layer1 = Dense(512, activation='relu')(input_layer)
    encode_layer2 = Dense(256, activation='relu')(encode_layer1)
    encode_layer3 = Dense(128, activation='relu')(encode_layer2)
    # latent view
    latent_view = Dense(64, activation='relu')(encode_layer3)
    # decoding architecture
    decode_layer1 = Dense(128, activation='relu')(latent_view)
    decode_layer2 = Dense(256, activation='relu')(decode_layer1)
    decode_layer3 = Dense(512, activation='relu')(decode_layer2)
    # output layer
    output_layer = Dense(784, activation='sigmoid')(decode_layer3)
    # model
    model = Model(input_layer, output_layer)
```

```
### Sparse Deep Net
def getDeepAE():
    # input layer
    input_layer = Input(shape=(784,))
    # encoding architecture
    encode_layer1 = Dense(1024, activation='relu')(input_layer)
    encode_layer2 = Dense(1024, activation='relu')(encode_layer1)
    # latent view
    latent_view = Dense(1024, activation='relu',
                        activity_regularizer=regularizers.l1(10e-5))(encode_layer2)
    # decoding architecture
    decode_layer1 = Dense(1024, activation='relu')(latent_view)
    decode_layer2 = Dense(1024, activation='relu')(decode_layer1)
    # output layer
    output_layer = Dense(784, activation='sigmoid')(decode_layer2)
    # model
    model = Model(input_layer, output_layer)

    return model
```

```
### ...
```

Relevant links:

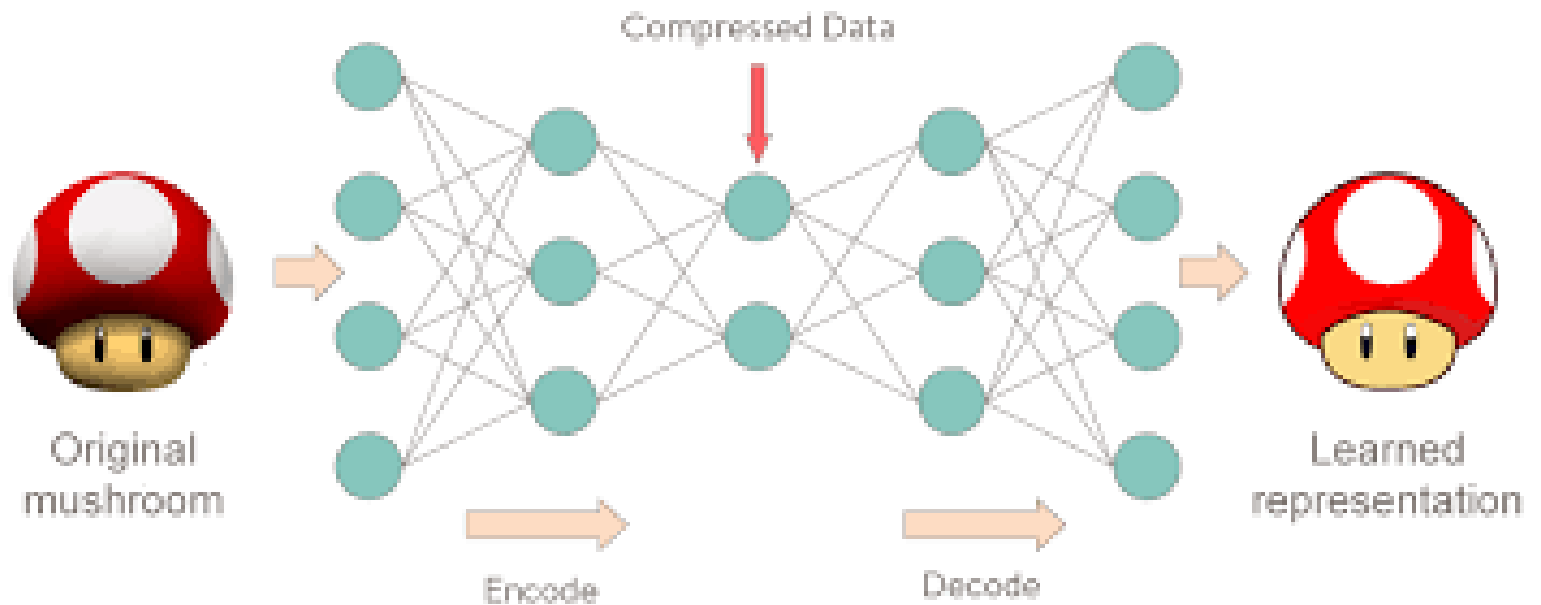
<https://towardsdatascience.com/implementing-an-autoencoder-in-tensorflow-2-0-5e86126e9f7>

<https://learnopencv.com/autoencoder-in-tensorflow-2-beginners-guide/>

<https://www.geeksforgeeks.org/contractive-autoencoder-cae/>

<https://medium.com/aimonks/contractive-autoencoders-an-insight-into-enhanced-feature-learning-d3d3bd103d88>

Autoencoders in Deep Learning



Feature extraction

Reconstruction / Generation

*Representation
transformation*

Convolutional Neural Networks (CNN)

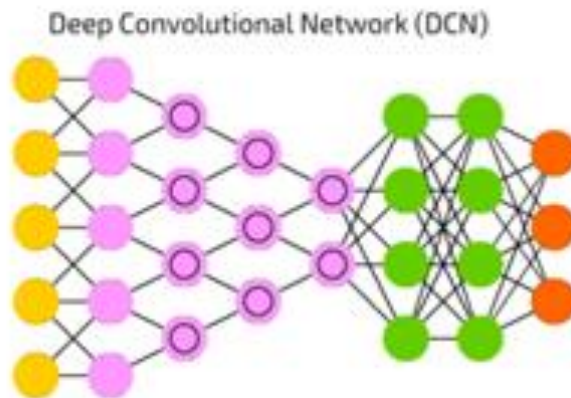


Image Classification



What We See

```
08 02 22 97 38 15 00 40 00 75 04 05 07 78 52 12 50 77 91 08
49 49 99 40 17 81 18 57 60 87 17 40 98 43 69 48 04 56 62 00
81 49 31 73 55 79 14 29 93 71 40 67 53 88 30 03 49 13 36 65
52 70 95 23 04 60 11 42 69 24 68 56 01 32 56 71 37 02 36 91
22 31 16 71 51 67 63 89 41 92 36 54 22 40 40 28 66 33 13 80
24 47 32 60 99 03 45 02 44 75 33 53 78 36 84 20 35 17 12 50
32 98 81 28 64 23 67 10 26 38 40 67 89 54 70 66 18 38 64 70
67 26 20 68 02 62 12 20 95 63 94 39 43 08 40 91 66 49 94 21
24 55 58 05 66 73 99 26 97 17 78 78 96 83 14 88 34 89 63 72
21 36 23 09 75 00 76 44 20 45 35 14 00 61 33 97 34 31 33 95
78 17 53 28 22 75 31 67 15 94 03 80 04 62 16 14 09 53 56 92
16 39 05 42 96 35 31 47 55 58 88 24 00 17 54 24 36 29 85 57
86 56 00 48 35 71 89 07 05 44 44 37 44 60 21 58 51 54 17 58
19 80 81 68 05 94 47 69 28 73 92 13 86 52 17 77 04 89 55 40
04 52 08 83 97 35 99 16 07 97 57 32 16 26 26 79 33 27 98 66
88 36 68 87 57 62 20 72 03 46 33 67 46 55 12 32 63 93 53 69
04 42 16 73 38 25 39 11 24 94 72 18 08 46 29 32 40 62 76 36
20 69 36 41 72 30 23 88 34 62 99 69 82 67 59 85 74 04 36 16
20 73 35 29 78 31 90 01 74 31 49 71 48 86 81 16 23 57 05 54
01 70 54 71 83 51 54 69 16 92 33 48 61 43 52 01 89 19 67 48
```

What Computers See

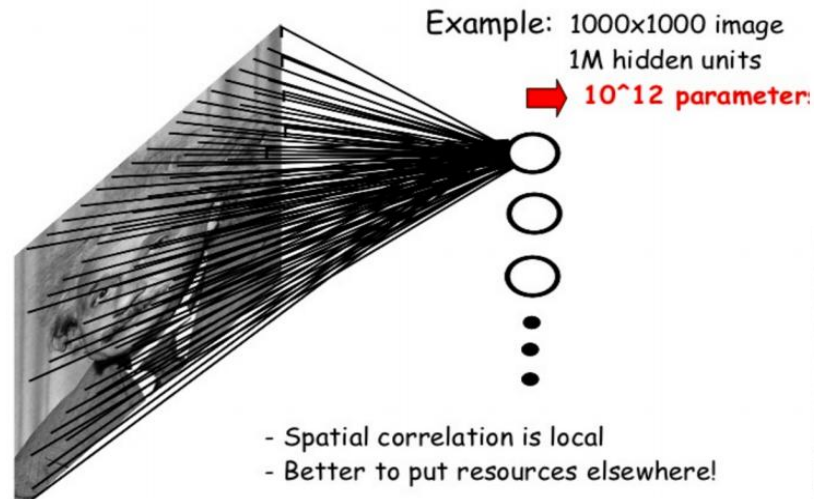
Our expectation: Computer takes an array of pixels ($w \times h \times 3$) with RGB values (0..255) and produces output number(s) that describe the probability of the image being a certain class (.80 for dog, .15 for cat, .05 for car, etc).

Image Classification

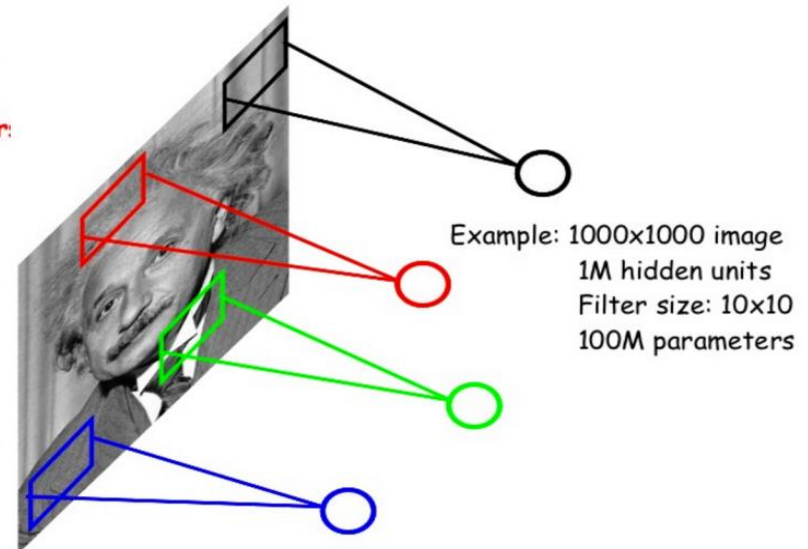
Fully connected DNN is very rarely used in computer vision tasks...

- Too many parameters require too much training data, computation power and time

FULLY CONNECTED NEURAL NET



LOCALLY CONNECTED NEURAL NET



- No generalization, no spatial invariance for images



CNNs

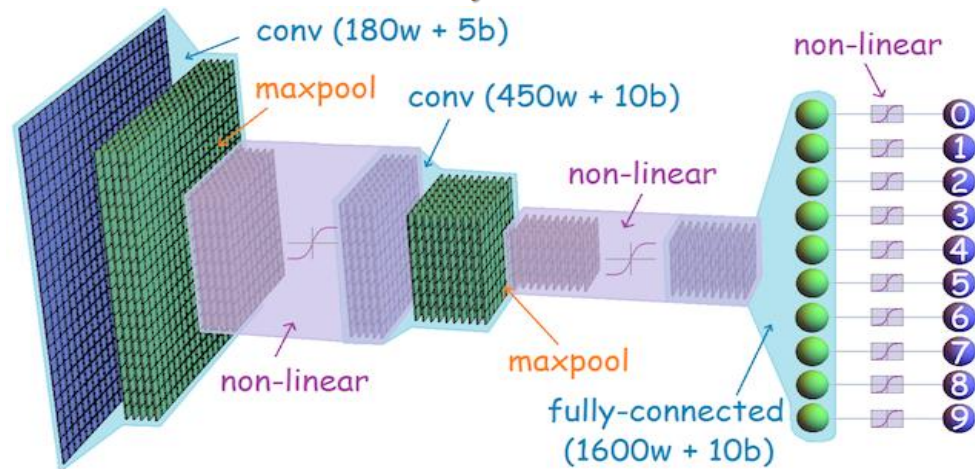
Approach to be used: In a similar way as human classify picture with a dog if it has identifiable features such as paws or 4 legs, the computer suppose to be able perform image classification by looking for low level features such as edges and curves, and then building up to more abstract concepts through a series of convolutional layers. Therefore, we have to train a computer to differentiate between all the images it's given and by figuring out the unique features that make a dog a dog or that make a cat a cat, etc.

Convolutional Neural Networks (CNN) are probably the most popular deep learning architecture for the moment for image processing...

CNN progressed from 8 layer AlexNet in 2012 towards 152 layer ResNet in 2015 and much further beyond...

Applied for:

- image related problem
- recommender systems
- natural language processing
- etc.



The main advantages:

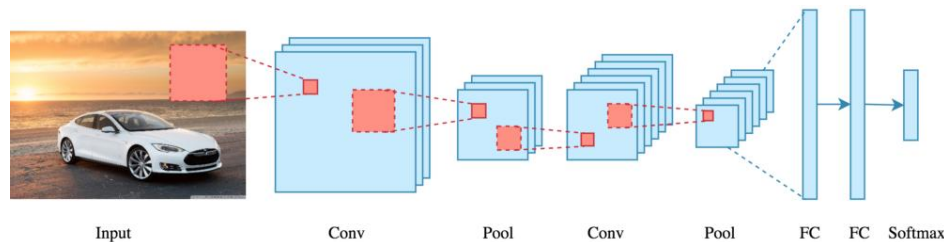
- Automatic detection of important features without any human supervision (e.g. given many pictures of two different classes it learns distinctive features for each class by itself).
- Using special convolution and pooling operations and parameter sharing, CNN becomes computationally efficient and enables models to be run on any device.

CNNs

CNN model is kind of a combination of two components:

Feature extraction. The convolution + pooling layers perform feature extraction and make possible to detect features like two eyes, four legs, road sign, two wheels, building, face, etc. The convolution layers learn these complex features by building on top of each other. The first layers detect edges, the next layers combine them to detect shapes, to following layers merge this information to infer complete objects.

Classification. The fully connected layers act as a classifier on top of the extracted features and assign a probability for the input image being a representative of certain class.



Relevant links:

<http://cs231n.stanford.edu/>

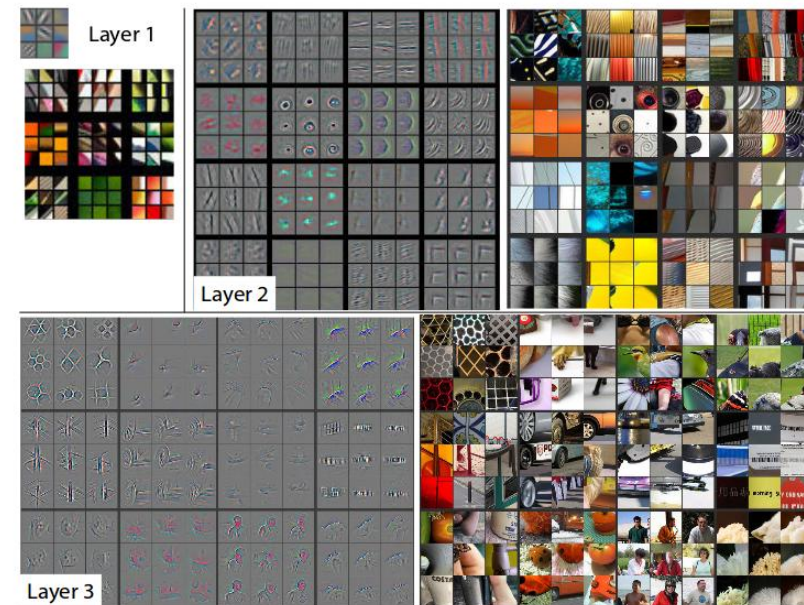
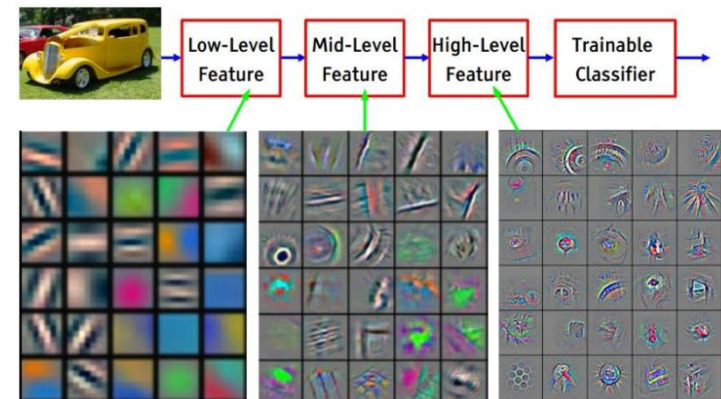
<http://cs231n.github.io/convolutional-networks/>

<http://www.matthewzeiler.com/wp-content/uploads/2017/07/arxiv2013.pdf>

<https://www.youtube.com/watch?v=AgkflQ4IGaM>

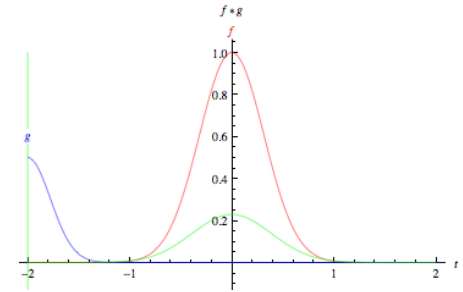
<https://towardsdatascience.com/applied-deep-learning-part-4-convolutional-neural-networks-584bc134c1e2>

<https://www.youtube.com/watch?v=2-OI7ZB0MmU>



Convolution

A convolution is an integral that expresses the amount of overlap of one function g as it is shifted over another function f . It therefore "blends" one function with another. In another words, **Convolution** - is a mathematical operation to merge two sets of information.

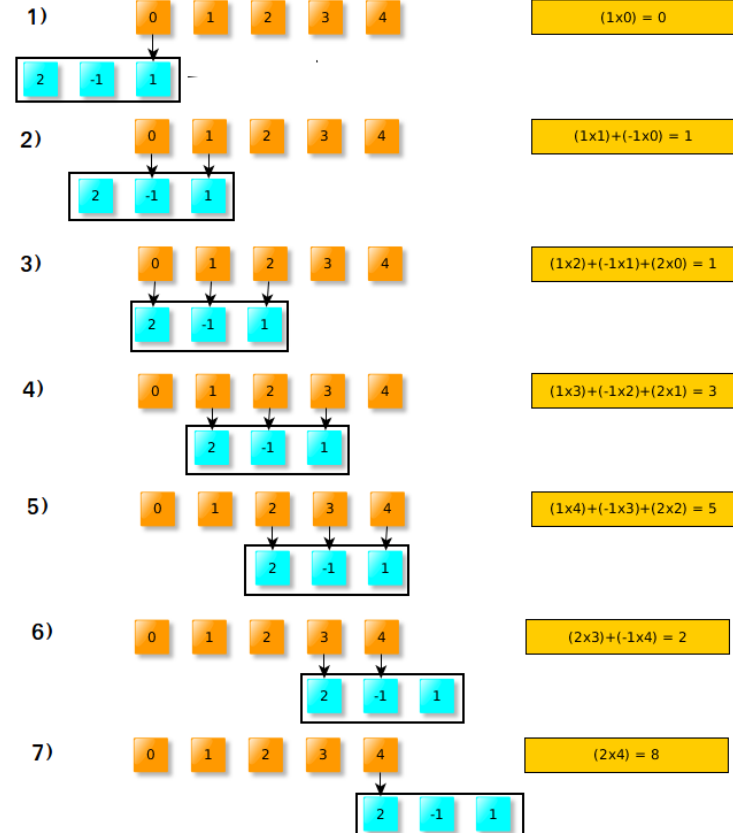


Convolution on signal processing is used for the following use cases:

- Filter signals (1D audio, 2D image processing)
- Check how much a signal is correlated to another
- Find patterns in signals



```
In [1]: import numpy as np
In [2]: x = np.array([0,1,2,3,4])
In [3]: w = np.array([1,-1,2])
In [4]: res = np.convolve(x,w)
In [5]: print res
[0 1 1 3 5 2 8]
```



Relevant links:

<https://en.wikipedia.org/wiki/Convolution>
<http://mathworld.wolfram.com/Convolution.html>
<https://leonardoaraujosantos.gitbooks.io/artificial-intelligence/content/convolution.html>

2D Convolution

2D convolutions are used as image filters, and when you would like to find a specific patch on an image.

We use a convolution filter on top of the input data to produce a feature map.

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Input

1	0	1
0	1	0
1	0	1

Filter / Kernel



*

1	0	-1
2	0	-2
1	0	-1



Convolution operation on matrixes (images context):

- slide the filter over the input
- do element-wise matrix multiplication and sum the result at every sliding location (receptive field)

1x1	1x0	1x1	0	0
0x0	1x1	1x0	1	0
0x1	0x0	1x1	1	1
0	0	1	1	0
0	1	1	0	0

Input x Filter

4		

Feature Map

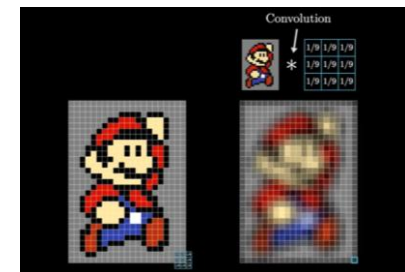


1	1x1	1x0	0x1	0
0	1x0	1x1	1x0	0
0	0x1	1x0	1x1	1
0	0	1	1	0
0	1	1	0	0

Input x Filter

4	3	

Feature Map

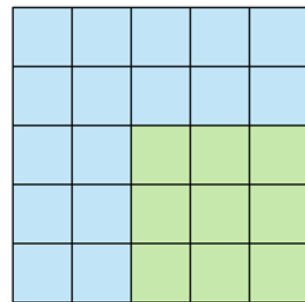


Nice visual explanation of Image Kernels: <http://setosa.io/ev/image-kernels/>

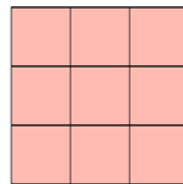
2D Convolution

Stride specifies how much we move the convolution filter at each step (e.g. stride 1 ($[1,1]$), stride 2 ($[2,2]$), etc.). We can have bigger strides if we want less overlap between the receptive fields making the resulting feature map smaller.

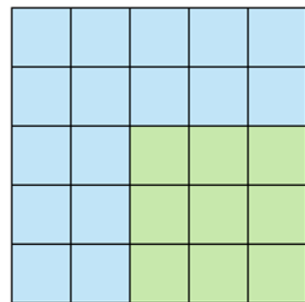
We can use **padding** to maintain the feature map's dimensionality similar to the input by surrounding input with zeros or the values on the edge.



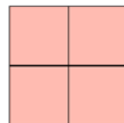
Stride 1



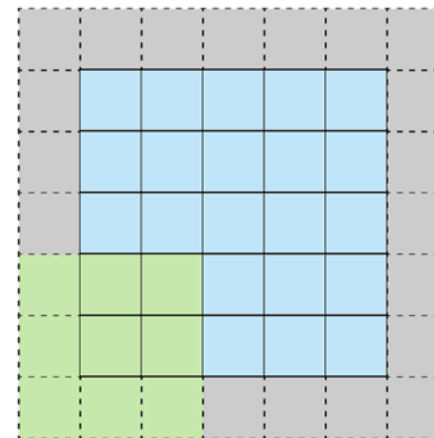
Feature Map



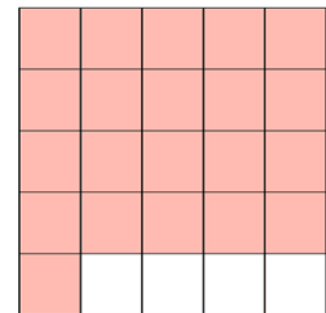
Stride 2



Feature Map



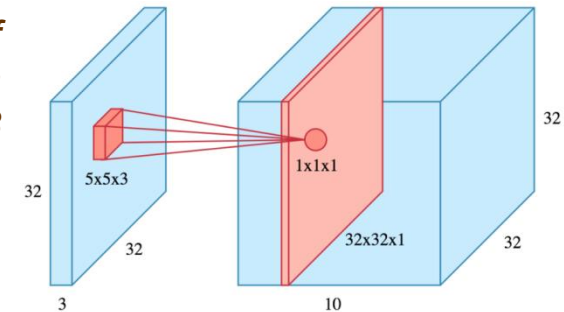
Stride 1 with Padding



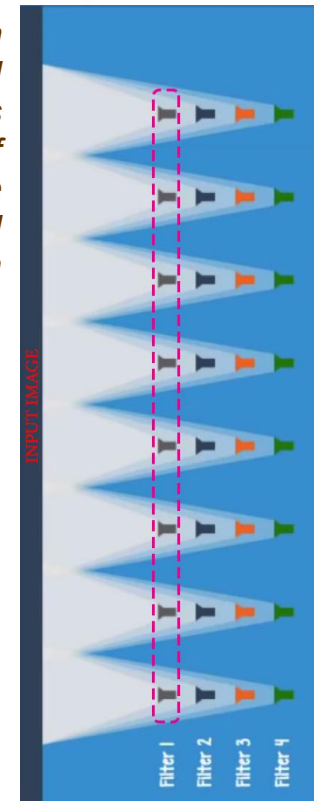
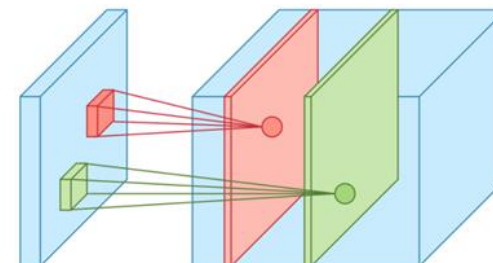
Feature Map

2D Convolution

Usually, an image is represented as a 3D matrix with dimensions of **height**, **width** and **depth** (color channels - RGB). In turn, a convolution filter become 3D as well having a specific height and width (e.g. 3x3 or 5x5), and by design covering the entire depth of the input.



Using different filters, we perform multiple convolutions on an input and stack all the distinct feature maps together forming the final output of the convolution layer. Each of these channels (distinct feature maps) will end up being trained to detect certain key features in the image (e.g. lines, edges or other distinctive shapes).



Input Volume (+pad 1) (7x7x3)	Filter W0 (3x3x3)	Filter W1 (3x3x3)	Output Volume (3x3x2)
$x[:, :, 0]$	$w0[:, :, 0]$	$w1[:, :, 0]$	$o[:, :, 0]$
0 0 0 0 0 0 0	-1 0 -1	-1 0 0	-1 -5 -5
0 1 1 0 2 0 0	-1 -1 1	0 0 1	2 -2 -8
0 2 2 2 2 1 0	-1 0 0	0 -1 1	-3 -7 -7
0 0 0 0 2 1 0	1 1 0	1 -1 1	3 0 -3
0 2 2 2 2 1 0	-1 -1 0	0 1 1	1 8 0
0 2 0 2 2 1 0	0 1 0	0 -1 -1	2 4 0
0 0 0 0 0 0 0	-1 1 0	0 0 -1	
$x[:, :, 1]$	$w0[:, :, 1]$	$w1[:, :, 1]$	$o[:, :, 1]$
0 0 0 0 0 0 0	-1 0 -1	-1 0 -1	
0 2 1 0 0 0 0	-1 0 -1	1 1 1	
0 0 2 0 1 0 0	-1 0 1		
0 1 0 1 2 0 0	1 1 0		
0 1 2 0 2 1 0	1 1 0		
0 1 2 1 2 2 0	1 1 0		
0 0 0 0 0 0 0	1 1 0		
$x[:, :, 2]$	$w0[:, :, 2]$	$w1[:, :, 2]$	
0 0 0 0 0 0 0	-1 1 0	0 0 -1	
0 2 1 1 2 2 0	-1 0 -1	-1 0 -1	
0 1 1 1 0 0 0	-1 0 1	1 1 1	
0 2 0 1 0 2 0	1 1 0		
0 0 2 0 2 1 0	1 1 0		
0 0 0 2 1 0 0	1 1 0		
0 0 0 0 0 0 0	1 1 0		

Bias b0 (1x1x1)
b0[:, :, 0]
1

Bias b1 (1x1x1)
b1[:, :, 0]
0

2D Convolution

To be powerful, neural network (that basically has just been computing linear operations during the conv layers) needs to contain **non-linearity**. It is achieved by passing the weighted sum of its inputs through an **activation function**.

In CNN we pass the result of the convolution operation through **ReLU** activation function, that works far better (comparably to **tanh** and **sigmoid**) and makes the network able to train a lot faster (due to the computational efficiency) without making a significant difference to the accuracy. So, the values in the final feature maps are not actually the sums, but the ReLU function applied to them.

Short summary on Convolution Layer:

There are **4 important hyper-parameters** to decide on:

- **Filter size.** **3x3 filters** are typically used, but 5x5 or 7x7 are also applicable depending on the application. Even 1x1 filters have sense, since filters are 3D and have a depth dimension as well.
- **Filters amount.** This is the most variable parameter, it's a **power of 2 anywhere between 32 and 1024**. More filters make a model more powerful, but we risk overfitting due to increased number of parameters. One of the strategies is to start with a small number of filters at the initial layers, and progressively increase number of them as go deeper into the network.
- **Stride.** Usually default value is 1 (**stride [1,1]**).
- **Padding.** Usually **use padding**.

Relevant links:

<http://www.cs.toronto.edu/~fritz/absps/reluICML.pdf>

https://en.wikipedia.org/wiki/Vanishing_gradient_problem

<https://www.quora.com/What-is-the-vanishing-gradient-problem>

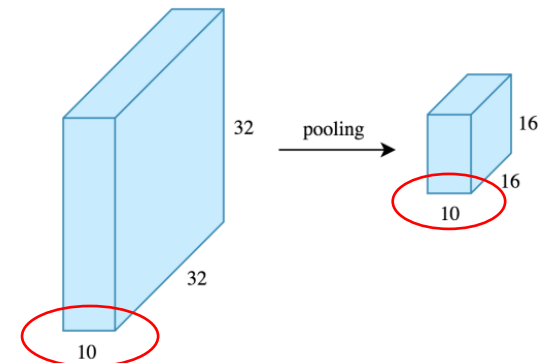
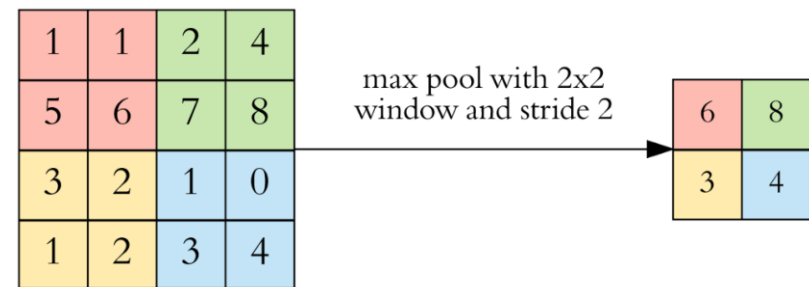
<https://www.youtube.com/watch?v=V9ZYDCnItr0>

Pooling

To reduce the number of parameters, which both **shortens the training time** and **combats overfitting**, we reduce the dimensionality of convolution operation result by performing a **pooling** (sub-sampling) operation. It is also a “sliding window” technique, that applies some sort of statistical function over the values within the window.

max pooling - the most common type of pooling operation that just slides a window over the input, and simply takes the max value in the window.

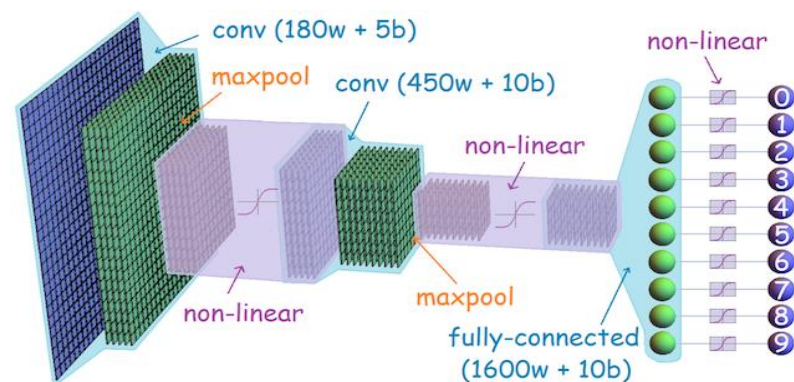
- There are some other alternatives e.g. mean (average) pooling or L2-norm pooling. Similar to a filter size and sliding step in convolution, we specify the **window size** and **stride** for pooling operation.
- Pooling layers reduce the height and width, keeping the depth of convolution layer output intact. It down-sample each feature map independently, while keeping the important information.
- The intuition behind the pooling layer is that once there is a high activation value, there will be a specific feature in the original input volume and its exact location is not as important as its relative location to the other features.



In CNN architectures, **pooling** is typically performed with **2x2 windows**, **stride 2** (stride [2,2]) and **no padding**. While **convolution** is done with **3x3 windows**, **stride 1** and **with padding**.

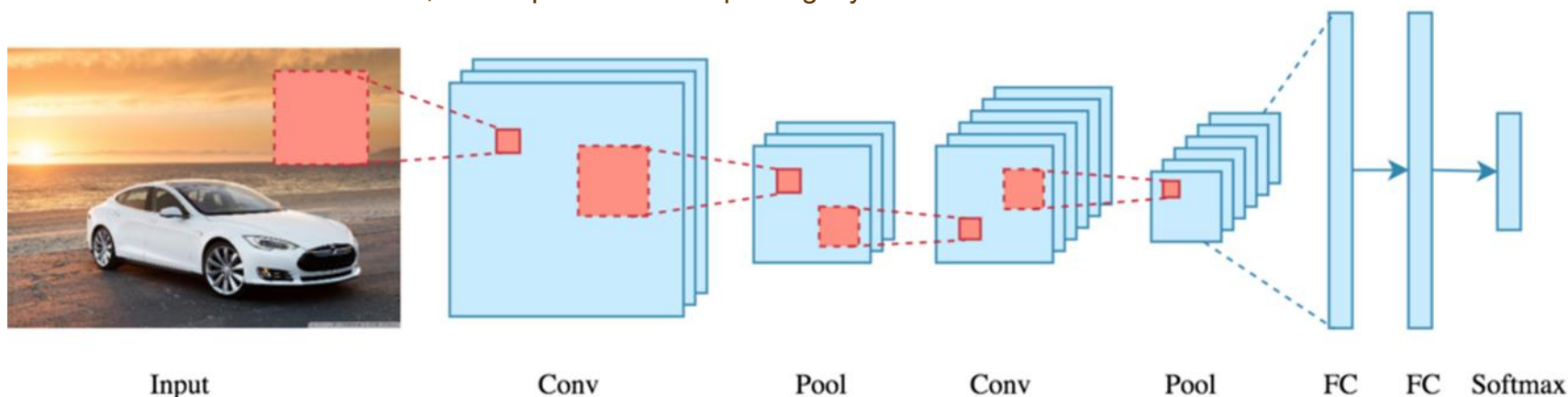
CNN

In CNN architecture, a chain of the *convolution + pooling* layers is wrapped up with a couple of *fully connected layers*.

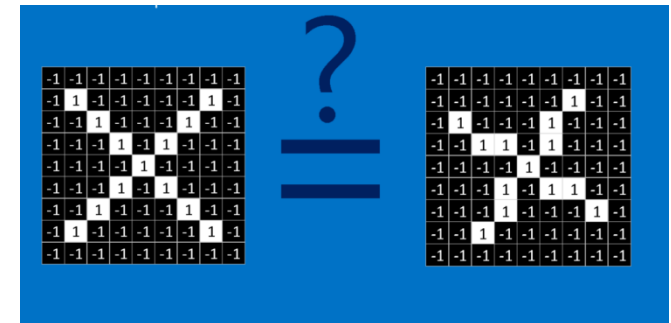
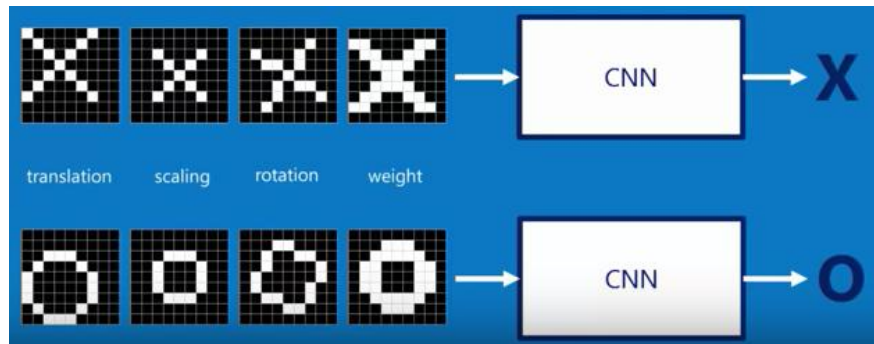


At the output of the convolutional-pooling layers we have moved *from high resolution low level data about the pixels to representations of objects within the image*. The purpose of these final, fully connected layers is to make classifications regarding these objects – so, we insert a standard neural network classifier onto the end of a trained object detector.

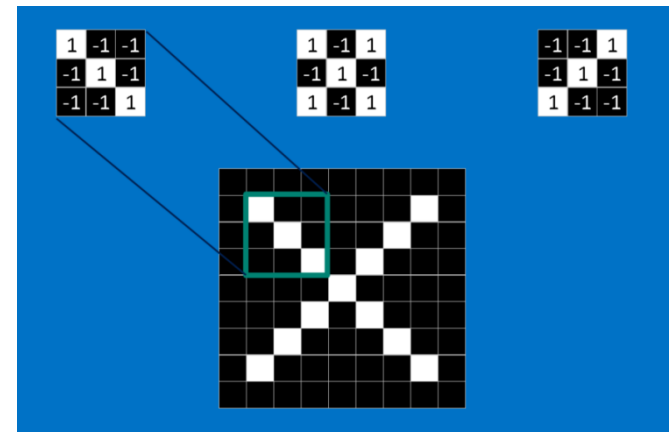
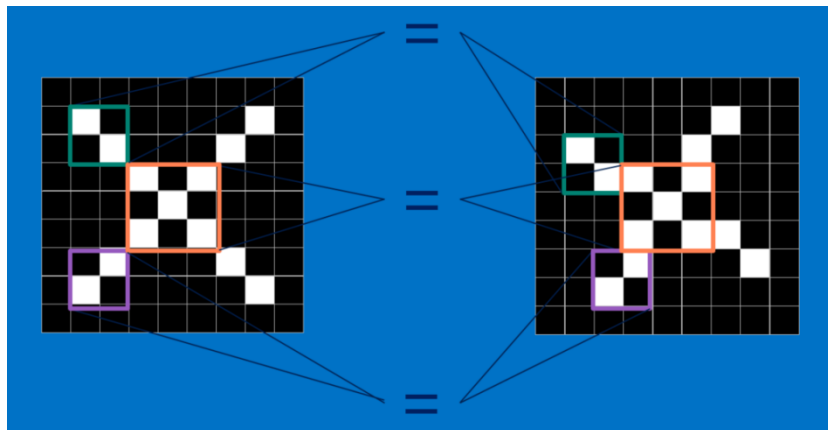
Since the output of both convolution and pooling layers are 3D volumes and a fully connected layer accepts input in a form of a 1D vector of numbers, the output of the final pooling layer should be *flatten* to a vector.



CNN simple example



*CNNs compare images based on generalized **features** rather than on pixel basis.
Comparing images piece by piece, CNNs try to find rough feature matches in roughly the same positions in two images.*



Relevant links:

https://brohrer.github.io/how_convolutional_neural_networks_work.html

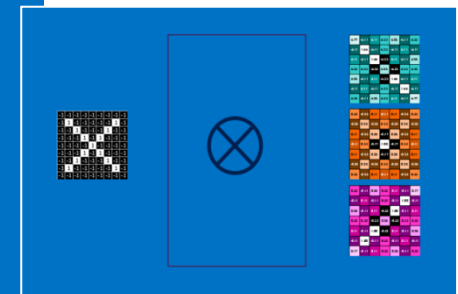
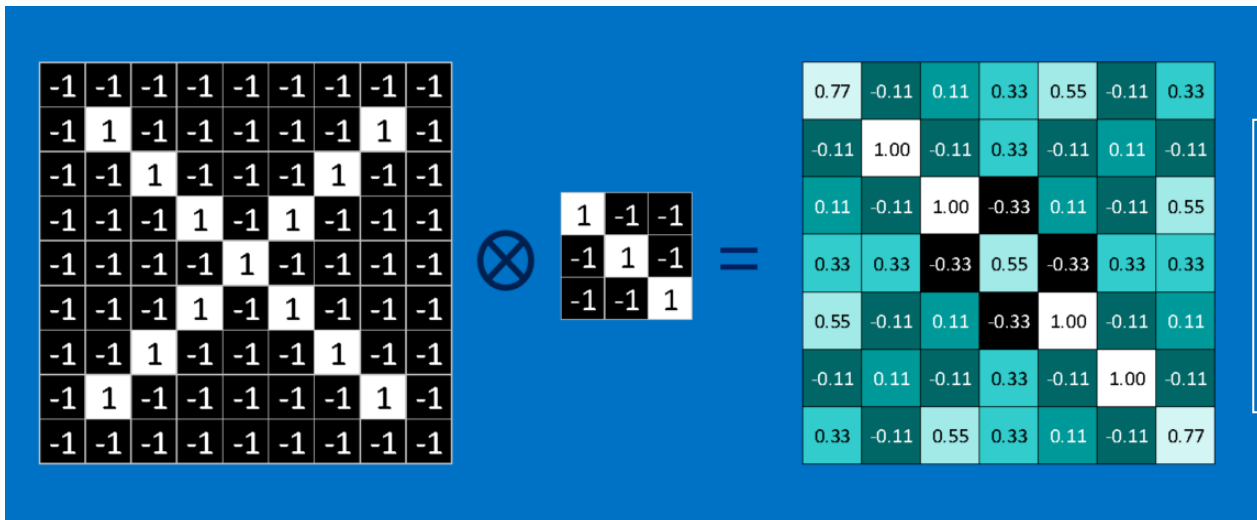
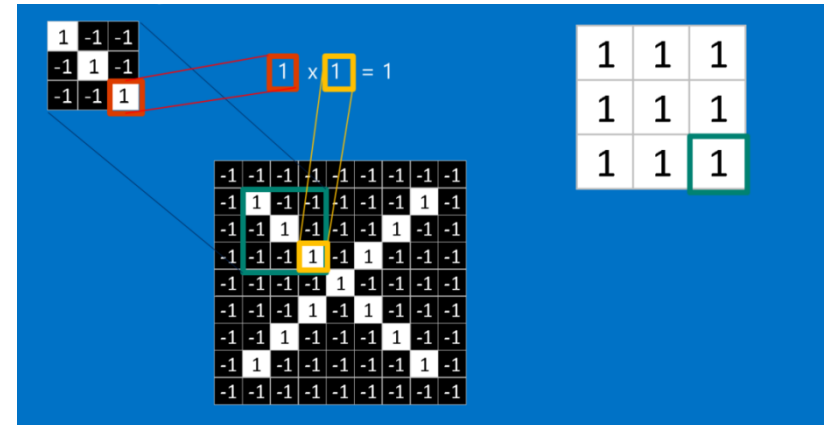
<https://www.edureka.co/blog/convolutional-neural-network/>

CNN simple example

Convolution makes a filtering of the image by calculating the match to a feature across it.

To calculate the match of a feature to a patch of the image, simply multiply each pixel in the feature by the value of the corresponding pixel in the image. Then add up the answers and divide by the total number of pixels in the feature.

Next repeat the convolution process in its entirety for each of the other features and get a set of filtered images, one for each of our filters.



Relevant links:

https://brohrer.github.io/how_convolutional_neural_networks_work.html

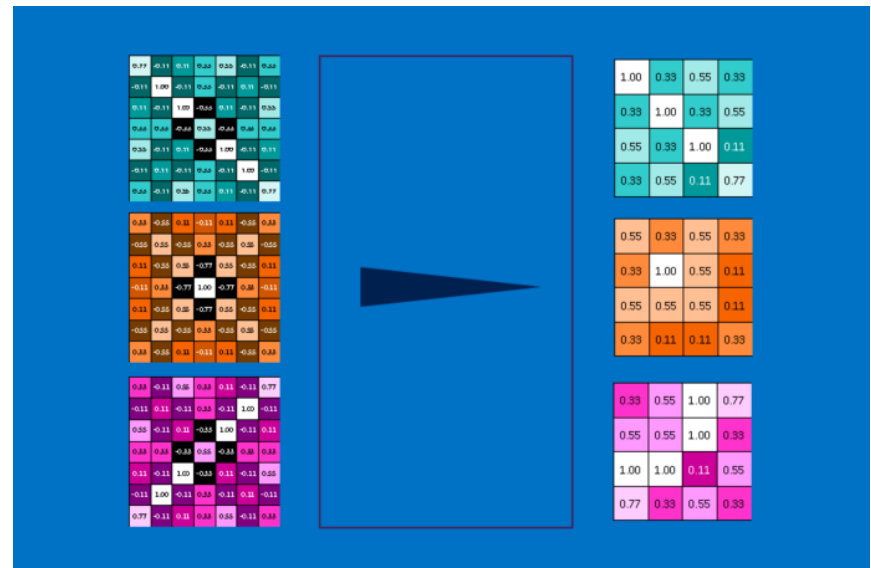
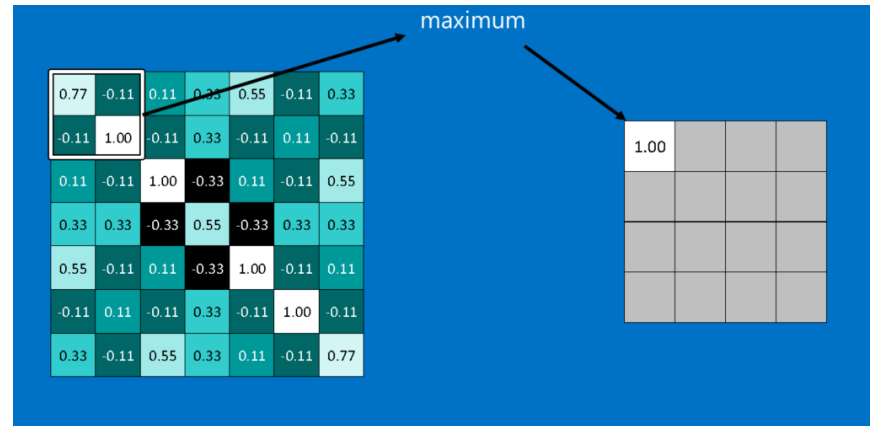
<https://www.edureka.co/blog/convolutional-neural-network/>

CNN simple example

Pooling is a way to take large images and shrink them down while preserving the most important information in them.

The math behind pooling consists of stepping a small window across an image and taking the maximum value (**max pooling**) from the window at each step.

After pooling, an image has about a quarter as many pixels as it started with. Because it keeps the maximum value from each window, it preserves the best fits of each feature within the window. This means that it doesn't care so much exactly where the feature fit as long as it fit somewhere within the window.



Relevant links:

https://brohrer.github.io/how_convolutional_neural_networks_work.html

<https://www.edureka.co/blog/convolutional-neural-network/>

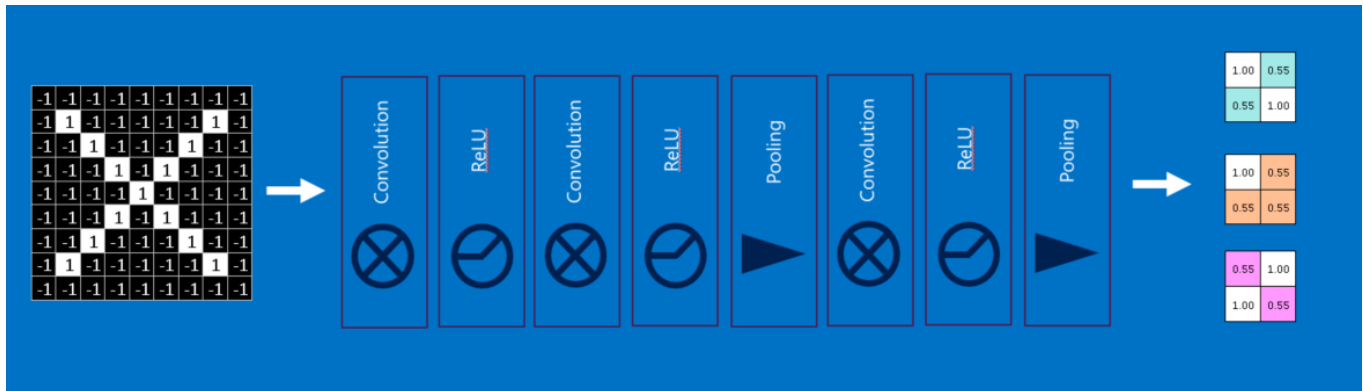
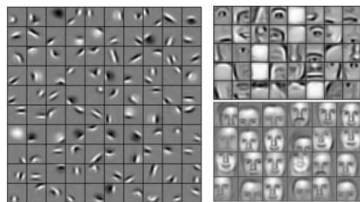
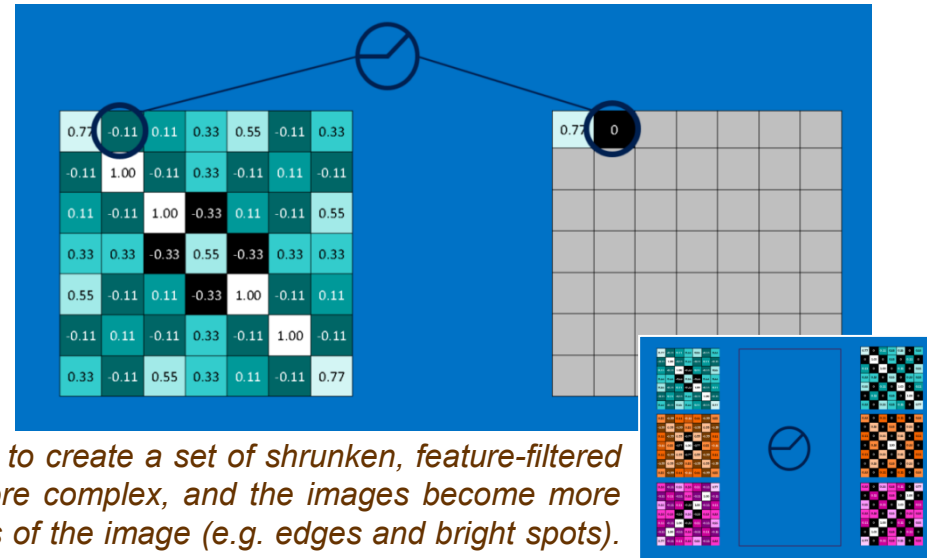
CNN simple example

Rectified Linear Unit (ReLU) helps the CNN stay mathematically healthy by keeping learned values from getting stuck near 0 or blowing up toward infinity.

Wherever a negative number occurs, swap it out for a 0

Layers could be stacked like Lego bricks.

As a result, raw images get filtered, rectified and pooled to create a set of shrunk, feature-filtered images. Each time, the features become larger and more complex, and the images become more compact. This lets lower layers represent simple aspects of the image (e.g. edges and bright spots). Higher layers can represent increasingly sophisticated aspects of the image, such as shapes and patterns.



Relevant links:

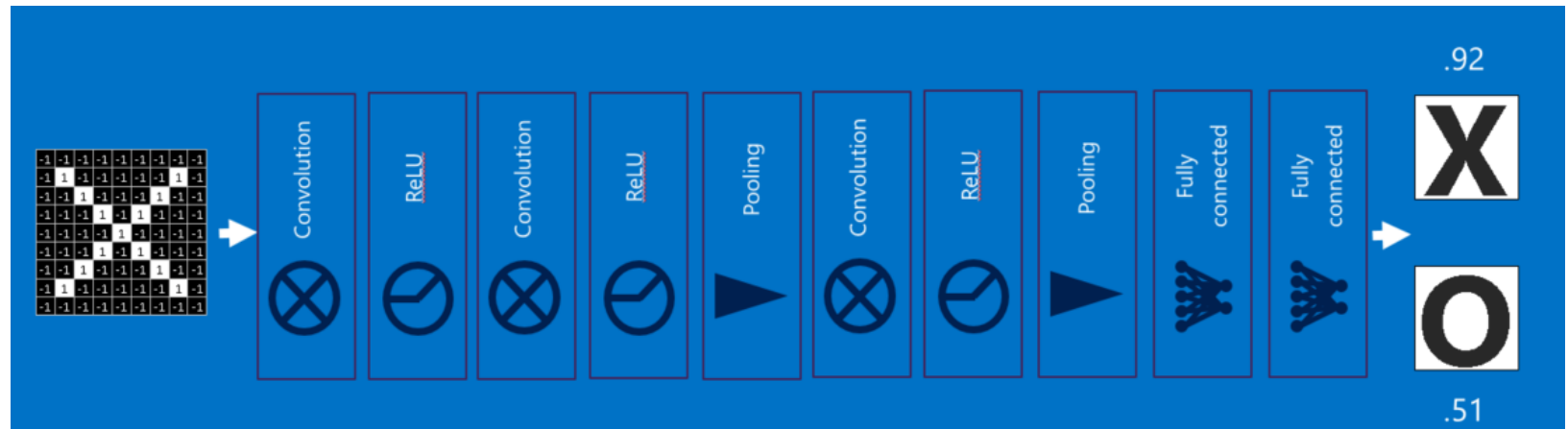
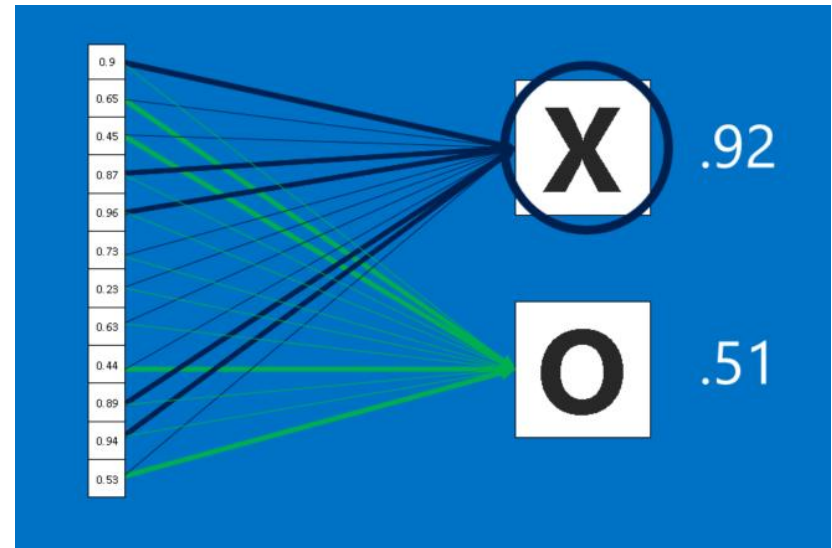
https://brohrer.github.io/how_convolutional_neural_networks_work.html

<https://www.edureka.co/blog/convolutional-neural-network/>

CNN simple example

Fully connected layers take the high-level filtered images and translate them into votes.

Instead of treating inputs as a two-dimensional array, Fully connected layers are treated as a single list and all treated identically. Every value gets its own vote on whether the current image is an X or an O. Some values are much better than others at knowing when the image is an X, and some are particularly good at knowing when the image is an O. These get larger votes than the others. These votes are expressed as weights, or connection strengths, between each value and each category.

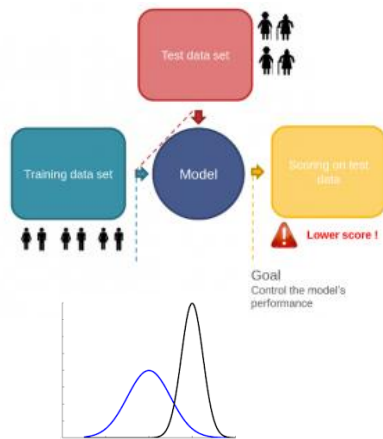


Relevant links:

https://brohrer.github.io/how_convolutional_neural_networks_work.html

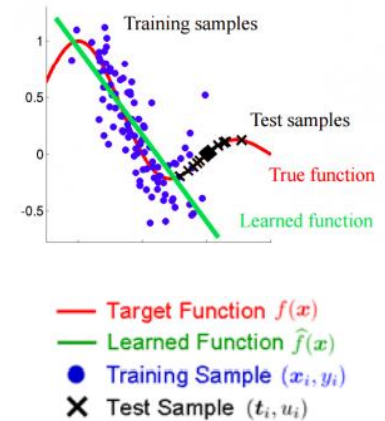
<https://www.edureka.co/blog/convolutional-neural-network/>

Normalization

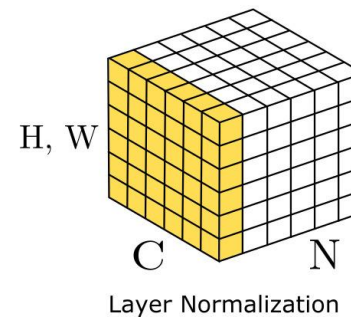
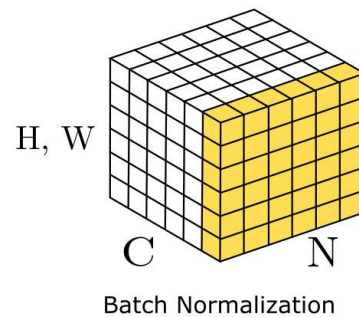


Change in the distribution of data contained in train and test file is called **dataset shift** (or **drifting**). For example, it could be shift in the independent variables called **Covariate Shift**. Normalization tackles this by stabilizing the learning process, ensuring that the model can adapt and learn effectively.

Normalize to optimize...



Normalization helps neural network to navigate the complex landscape of training and avoid getting stuck in suboptimal performance. Therefore, **Batch Normalization** and **Layer Normalization** steer a model towards faster and more reliable convergence...



Relevant links:

<https://medium.com/biased-algorithms/batch-normalization-vs-layer-normalization-c44472883bf2>

<https://www.analyticsvidhya.com/blog/2017/07/covariate-shift-the-hidden-problem-of-real-world-data-science/>

Normalization

Batch Normalization (BN) is designed to normalize the input of each layer within a neural network across the mini-batch. This means that during training, each layer receives data that is centered (mean of zero) and scaled (unit variance), making the learning process smoother and more efficient.

Normalization: It calculates the mean and variance for each feature across the current mini-batch.

Scaling and Shifting: After normalizing the data, it applies a scaling factor (gamma) and a shifting factor (beta). These are learnable parameters, meaning the network can optimize them during training for better performance.

```
tf.keras.layers.BatchNormalization(axis=-1, momentum=0.99, epsilon=0.001,
center=True, scale=True, beta_initializer='zeros', gamma_initializer='ones',
moving_mean_initializer='zeros', moving_variance_initializer='ones',
beta_regularizer=None, gamma_regularizer=None, beta_constraint=None,
gamma_constraint=None,
synchronized=False, **kwargs)
```

Relevant links:

<https://theaisummer.com/normalization/>

<https://paperswithcode.com/method/layer-normalization>

<https://arxiv.org/abs/1803.08494>

<https://medium.com/biased-algorithms/batch-normalization-vs-layer-normalization-c44472883bf2>

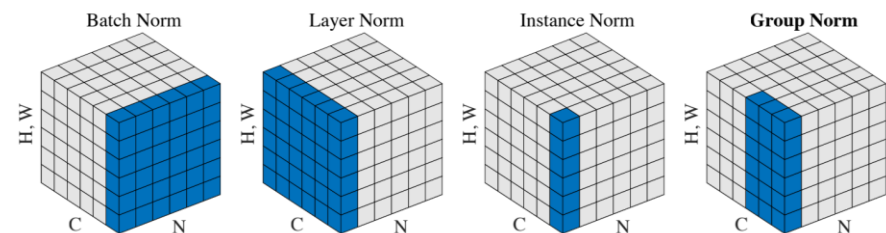
<https://www.youtube.com/watch?v=yXOMHOpon8>

Advantages:

- **Faster Convergence:** the network trains faster since each layer receives more stable data.
- **Regularization Effect:** by introducing a slight noise due to mini-batch variation, Batch Normalization has a regularizing effect, reducing the need for other techniques like dropout.
- **Higher Learning Rates:** since Batch Normalization reduces internal covariate shift, it becomes possible to use higher learning rates, making the training process more efficient.

Limitations:

- **Dependent on Mini-Batch Size:** in case of too small size of mini-batch, the statistics (mean and variance) might not be reliable, leading to suboptimal performance.
- **Not Suitable for Recurrent Networks:** since the input sequence length of RNN varies, Batch Normalization is not as effective due to the lack of consistent mini-batches.



```
tf.keras.layers.LayerNormalization(axis=-1, epsilon=0.001, center=True, scale=True, rms_scaling=False,
beta_initializer='zeros', gamma_initializer='ones', beta_regularizer=None, gamma_regularizer=None,
beta_constraint=None, gamma_constraint=None, **kwargs)
```

Normalization

Layer Normalization (LN) (unlike Batch Normalization, which normalizes across the batch) normalizes across the features of each individual data point (each data point gets normalized independently, irrespective of the batch size). Therefore, it becomes useful in models where batch size varies or is very small (for example, for RNNs or transformers where input sequences can vary in length).

In some other cases, e.g. Reinforcement Learning (where the data distribution can significantly change over time while agent learns and explores the environment) or in case of Few-Shot Learning (where a very small number of examples is used for a model training) where both BN and LN might be less effective and will not provide the desired benefits, some other alternatives might be used:

Group Normalization (GN): Normalizes across groups of channels and is effective for smaller batch sizes.

Instance Normalization (IN): Normalizes each instance in a batch independently, useful in style transfer tasks..

Relevant links:

<https://theaisummer.com/normalization/>

<https://paperswithcode.com/method/layer-normalization>

<https://arxiv.org/abs/1803.08494>

<https://medium.com/biased-algorithms/batch-normalization-vs-layer-normalization-c44472883bf2>

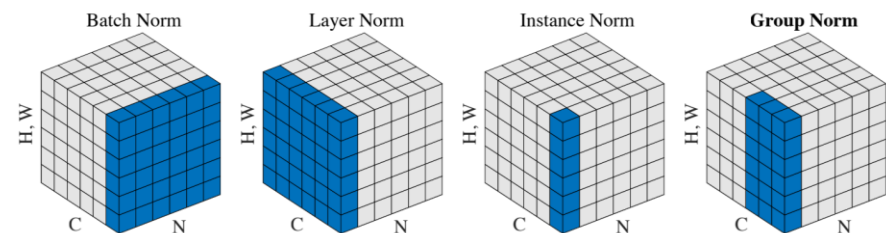
<https://www.youtube.com/watch?v=2V3Uduw1zwQ>

Advantages:

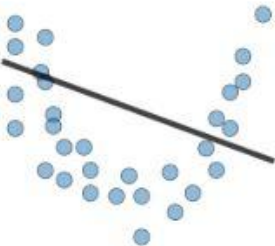
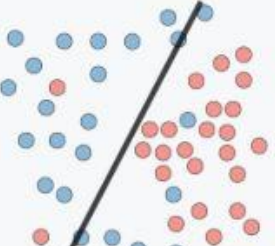
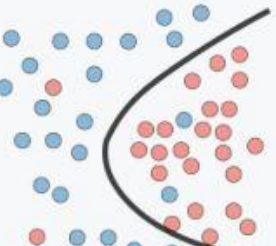
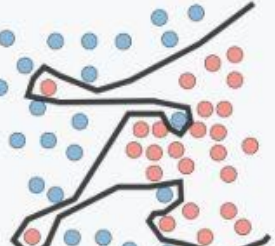
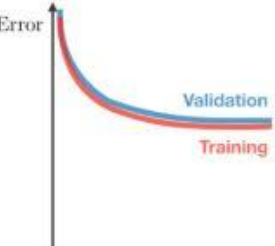
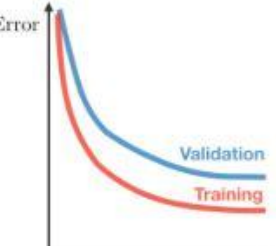
- **Independence from Batch Size:** regardless of how many samples are in a batch, Layer Normalization makes it perfect for tasks where batch size is limited or varies.
- **Better for RNNs and Transformers:** Layer Normalization becomes more effective for sequence data or NLP tasks, comparably to Batch Normalization.
- **Effective in Sequence Modeling:** for the models that deal with long-term dependencies, Layer Normalization stabilizes the learning process.

Limitations:

- **Less Effective in CNNs:** Batch Normalization tends to outperform Layer Normalization in convolutional networks, where spatial invariance and batch-based learning are crucial.
- **Computational Cost:** in some cases, Layer Normalization can be more computationally intensive due to normalization over a larger set of parameters.

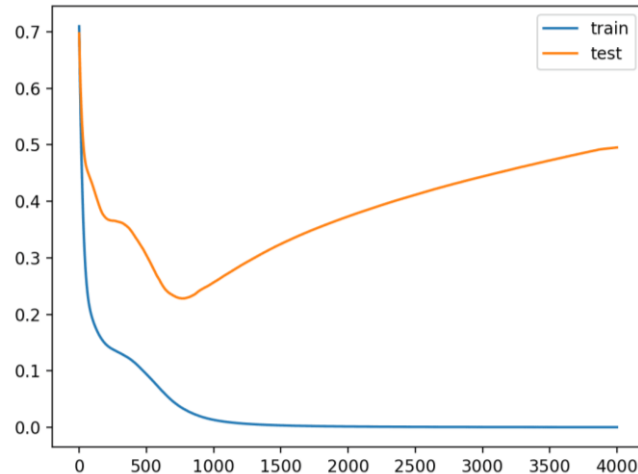


Model fitting

	Underfitting	Just right	Overfitting
Symptoms	• High training error • Training error close to test error • High bias	• Training error slightly lower than test error	• Very low training error • Training error much lower than test error • High variance
Regression illustration			
Classification illustration			
Deep learning illustration			
Possible remedies	• Complexify model • Add more features • Train longer		• Perform regularization • Get more data

Regularization

Too many epochs can lead to overfitting of the training dataset, whereas too few may result in an underfit model...



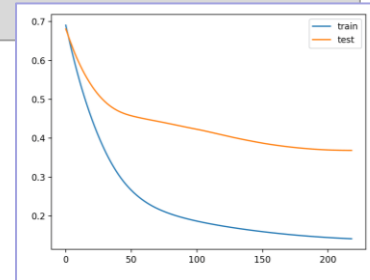
Early stopping is a method that allows use of an arbitrary large number of training epochs and stop training once the model performance stops improving.

```
tf.keras.callbacks.EarlyStopping(  
    monitor='val_loss',  
    min_delta=0,  
    patience=0,  
    verbose=0,  
    mode='auto',  
    baseline=None,  
    restore_best_weights=False,  
    start_from_epoch=0  
)
```

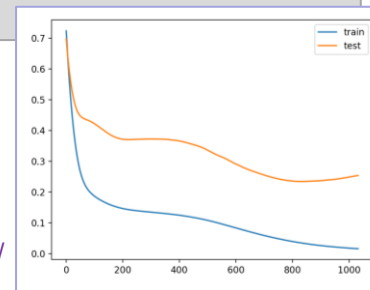
Relevant links:

<https://machinelearningmastery.com/how-to-stop-training-deep-neural-networks-at-the-right-time-using-early-stopping/>
<https://www.youtube.com/watch?v=EehRcPo1M-Q>

```
from tensorflow.keras.callbacks import EarlyStopping  
...  
# simple early stopping  
es = EarlyStopping(monitor='val_loss', mode='min', verbose=1)  
# fit model  
history = model.fit(trainX, trainy, validation_data=(testX, testy),  
                    epochs=4000, verbose=1, callbacks=[es])  
...
```



```
from tensorflow.keras.callbacks import EarlyStopping  
...  
# simple early stopping  
es = EarlyStopping(monitor='val_loss', mode='min', verbose=1,  
                  patience=200)  
# fit model  
history = model.fit(trainX, trainy, validation_data=(testX, testy),  
                    epochs=4000, verbose=1, callbacks=[es])  
...
```



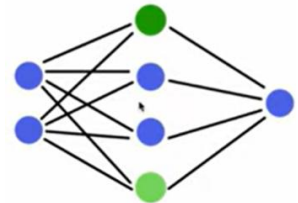
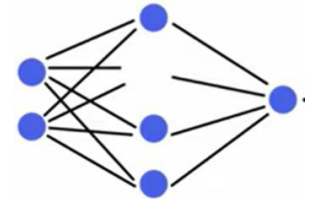
Regularization

Overfitting can be addressed by limiting the flexibility of the model (keeping it general enough) via lowering the model's weights...

The idea of **L1&L2 Regularization** is to penalizing the model for high weights by adding them into the loss calculation.

L1 regularization (or Lasso Regression) makes the network sparse... By adding the sum of the weights to the loss, L1 encourages the network to have the weights as low as 0, leading to the case when some of the outputs of the layer will not be considered at all.

$$\sum_{i=1}^n (Y_i - \sum_{j=1}^p X_{ij} \beta_j)^2 + \lambda \sum_{j=1}^p |\beta_j|$$



L2 regularization (or ridge regression or weight decay) adds the sum of the squared values of the weights to the loss. As a result, we have less sparse weights, since L2 penalizes larger weights more severely.

$$\sum_{i=1}^n (y_i - \sum_{j=1}^p x_{ij} \beta_j)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

```
from tensorflow.keras import layers
from tensorflow.keras import regularizers
```

```
layer = layers.Dense(
    units=64,
    kernel_regularizer=regularizers.L1L2(l1=1e-5, l2=1e-4),
    bias_regularizer=regularizers.L2(1e-4),
    activity_regularizer=regularizers.L2(1e-5)
)
```

Relevant links:

<https://keras.io/api/layers/regularizers/>

<https://www.youtube.com/watch?v=EehRcPo1M-Q>

<https://builtin.com/data-science/l2-regularization>

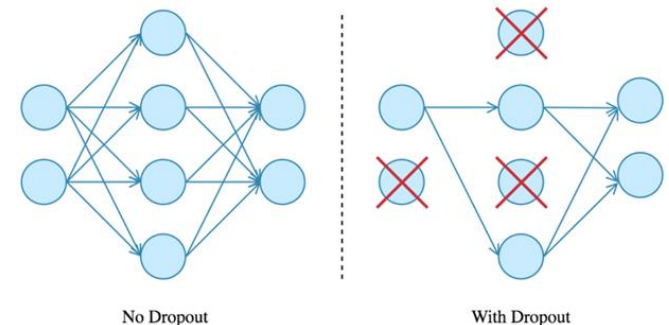
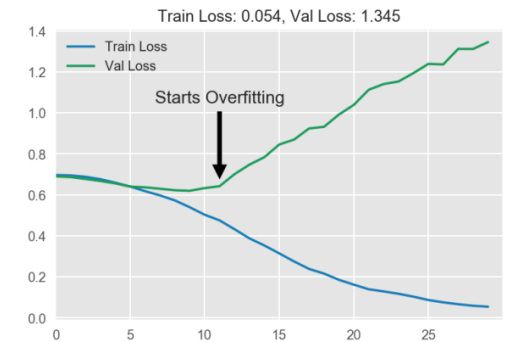
<https://medium.com/@alejandro.itoaramendia/l1-and-l2-regularization-part-1-a-complete-guide-51cf45bb4ade>

Regularization

We may face a case, when training loss keeps going down, but the validation loss starts increasing after certain epoch. It is a sign of **overfitting**. It tells us that our model is memorizing the training data, but it's failing to generalize to new instances.

The most popular regularization technique for deep neural networks is **Dropout**. It is used to prevent overfitting via temporarily “dropping”/disabling a neuron with probability p (a hyperparameter called the **dropout-rate** that is typically a number around 0.5) at each iteration during the training phase.

- The dropped-out neurons are resampled with probability p at every training step (a dropped out neuron at one step can be active at the next one). (Dropout is not applied during test time after the network is trained).
- Dropout can be applied to input or hidden layer nodes but not the output nodes.



Reason. Dropout forces every neuron to be able to operate independently and prevents the network to be too dependent on a small number of neurons.

Relevant links:

<https://www.tensorflow.org/tutorials/images/classification>

<https://www.cs.toronto.edu/~hinton/absps/JMLRdropout.pdf>

<https://arxiv.org/abs/1502.03167>

<https://www.youtube.com/watch?v=EehRcPo1M-Q>

Regularization

Overfitting happens when we are training on very few examples (e.g. 1000 images per category), and even use of dropout does not help us much. To start think about Deep Learning, we have to operate with at least 100K training examples. On a small dataset we will overfit no matter which regularization technique is applied.

Fortunately, there is a **data augmentation** method which enables us to train deep models on small datasets. It allows us to artificially boost the size of the training set through enrichment or “augmentation” of the training data by generating new examples via random transformation of existing ones.

Data augmentation is done dynamically during training time. To generate realistic images, the common transformations are: rotation, shifting, resizing, exposure adjustment, contrast change etc.



Also, some data cleaning tricks on images can be applied (e.g. whitening and mean normalization).

<http://ufldl.stanford.edu/tutorial/unsupervised/PCAWWhitening/>



Relevant links:

<https://www.tensorflow.org/tutorials/images/classification>

<https://keras.io/preprocessing/image/>

<https://machinelearningmastery.com/image-augmentation-deep-learning-keras/>

<https://www.youtube.com/watch?v=EehRcPo1M-Q>



Keras

Convolution NN on MNIST dataset with Keras



```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
get_ipython().magic(u'matplotlib inline')
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Activation,
    Flatten, BatchNormalization, Conv2D, MaxPooling2D,
    ZeroPadding2D, GlobalAveragePooling2D
from tensorflow.keras.optimizers import Adam
from tensorflow.keras import utils
from tensorflow.keras.preprocessing.image import
    ImageDataGenerator

# Import MNIST data
(X_train, Y_train), (X_test, Y_test) = mnist.load_data()
X_train = X_train.reshape(X_train.shape[0], 28, 28, 1)
X_test = X_test.reshape(X_test.shape[0], 28, 28, 1)
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')
X_train /= 255
X_test /= 255
# convert output if use 'categorical_crossentropy', or do not, if
# 'sparse_categorical_crossentropy'
#Y_train = utils.to_categorical(y_train, number_of_classes)
#Y_test = utils.to_categorical(y_test, number_of_classes)
number_of_classes = 10

# Three steps to create a CNN
# 1. Convolution
# 2. Activation
# 3. Pooling
```

```
# Repeat Steps 1,2,3 for adding more hidden layers
# 4. After that make a fully connected network
# This fully connected network gives ability to
# the CNN to classify the samples
model = Sequential()
```

```
model.add(Conv2D(32, (3, 3), input_shape=(28,28,1)))
model.add(BatchNormalization(axis=-1))
model.add(Activation('relu'))
model.add(Conv2D(32, (3, 3)))
model.add(BatchNormalization(axis=-1))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2,2)))
```

```
model.add(Conv2D(64,(3, 3)))
model.add(BatchNormalization(axis=-1))
model.add(Activation('relu'))
model.add(Conv2D(64, (3, 3)))
model.add(BatchNormalization(axis=-1))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2,2)))
```

```
model.add(Flatten())
# Fully connected layer
model.add(Dense(512))
model.add(BatchNormalization())
model.add(Activation('relu'))
model.add(Dropout(0.2))
model.add(Dense(number_of_classes))
```



Keras

Convolution NN on MNIST dataset with Keras

```
#model.compile(loss=tf.keras.losses.categorical_crossentropy, optimizer=Adam(), metrics=['accuracy'])
model.compile(loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True), optimizer=Adam(), metrics=['accuracy'])

gen = ImageDataGenerator(rotation_range=8, width_shift_range=0.08, shear_range=0.3,
                          height_shift_range=0.08, zoom_range=0.08)

test_gen = ImageDataGenerator()

train_generator = gen.flow(X_train, Y_train, batch_size=64)
test_generator = test_gen.flow(X_test, Y_test, batch_size=64)

model.fit(train_generator, batch_size=64, epochs=5, validation_data=test_generator)
```



The **CIFAR-10** and **CIFAR-100** are labeled subsets of the 80 million tiny images dataset, collected by Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton.

<https://www.cs.toronto.edu/~kriz/cifar.html>

CNN on CIFAR-10 tutorial: <https://www.tensorflow.org/tutorials/images/cnn>



Some datasets: http://rodrigob.github.io/are_we_there_yet/build/classification_datasets_results.html

MNIST CNN based on csv data: <https://github.com/tgjeon/kaggle-MNIST/blob/master/mnist.py>

Visualizing convolutional neural networks (CNN with colored images): <https://www.oreilly.com/ideas/visualizing-convolutional-neural-networks>

Relevant links:

<https://elitedatascience.com/keras-tutorial-deep-learning-in-python>

<http://cv-tricks.com/tensorflow-tutorial/keras/>

<http://adventuresinmachinelearning.com/keras-tutorial-cnn-11-lines/>



Keras

CNN on Fashion-MNIST dataset...

```
from __future__ import print_function
import tensorflow as tf
from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Conv2D,
    MaxPooling2D, Flatten
from tensorflow.keras.optimizers import RMSprop, SGD, Adam
import matplotlib.pyplot as plt
%matplotlib inline
```

```
batch_size = 128
num_classes = 10
epochs = 20
```

```
# Data Preparation
```

```
img_rows, img_cols = 28, 28
```

```
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
```

```
if tf.keras.backend.image_data_format() == 'channels_first':
```

```
    x_train = x_train.reshape(x_train.shape[0], 1, img_rows, img_cols)
```

```
    x_test = x_test.reshape(x_test.shape[0], 1, img_rows, img_cols)
```

```
    input_shape = (1, img_rows, img_cols)
```

```
else:
```

```
    x_train = x_train.reshape(x_train.shape[0], img_rows, img_cols, 1)
```

```
    x_test = x_test.reshape(x_test.shape[0], img_rows, img_cols, 1)
```

```
    input_shape = (img_rows, img_cols, 1)
```

```
x_train = x_train.astype('float32')
```

```
x_test = x_test.astype('float32')
```

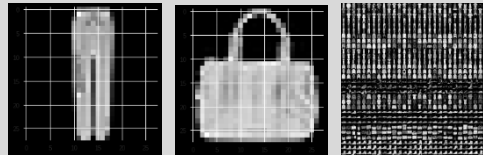
```
x_train /= 255
```

```
x_test /= 255
```

```
print('x_train shape:', x_train.shape)
```

```
print(x_train.shape[0], 'train samples')
```

```
print(x_test.shape[0], 'test samples')
```



```
# convert class vectors to binary class matrices if use
# 'categorical_crossentropy' loss. Otherwise use
# 'sparse_categorical_crossentropy' loss.
y_train = tf.keras.utils.to_categorical(y_train, num_classes)
y_test = tf.keras.utils.to_categorical(y_test, num_classes)
```

```
# Model Building
```

```
model = Sequential()
```

```
model.add(Conv2D(32, kernel_size=(3, 3),
    activation='relu',
    input_shape=input_shape))
```

```
model.add(Conv2D(64, (3, 3), activation='relu'))
```

```
model.add(MaxPooling2D(pool_size=(2, 2)))
```

```
model.add(Dropout(0.25))
```

```
model.add(Flatten())
```

```
model.add(Dense(128, activation='relu'))
```

```
model.add(Dropout(0.5))
```

```
model.add(Dense(num_classes, activation='softmax'))
```

```
model.compile(loss=tf.keras.losses.categorical_crossentropy,
    optimizer=Adam(), metrics=['accuracy'])
```

```
model.fit(x_train, y_train,
```

```
    batch_size=batch_size,
```

```
    epochs=epochs,
```

```
    verbose=1,
```

```
    validation_data=(x_test, y_test))
```

```
score = model.evaluate(x_test, y_test, verbose=0)
```

```
print('Test loss:', score[0])
```

```
print('Test accuracy:', score[1])
```

Label Description

- 0 T-shirt
- 1 Trouser
- 2 Pullover
- 3 Dress
- 4 Coat
- 5 Sandal
- 6 Shirt
- 7 Sneaker
- 8 Bag
- 9 Ankle boot

```
...
Epoch 20/20
55040/60000 [=====...] - ETA: 0s - loss: 0.1247 - acc:
0.955060000/60000 [=====] - 10s 165us/step - loss:
0.1239 - acc: 0.9553 - val_loss: 0.2282 - val_acc: 0.9299
Test loss: 0.22820208635628222
Test accuracy: 0.9299
```